

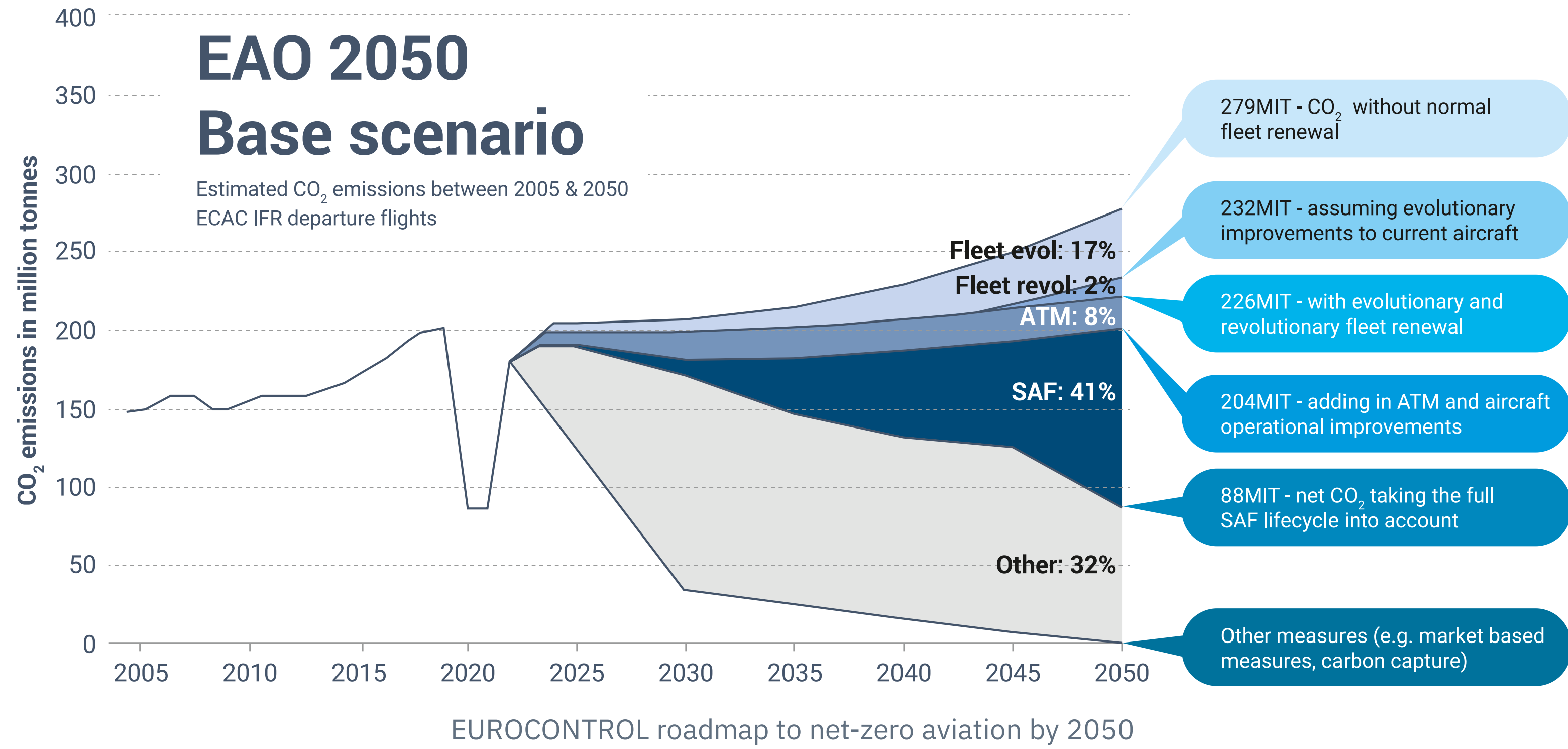
Leveraging Open Data and Models to Reduce Aviation's Climate Footprint

ISA Research School · May 2026

<https://github.com/junzis/isa-research-school-2026>

Junzi Sun · TU Delft

Background and context: aviation decarbonization



Background and context: operational efficiency



SUPPORTING
EUROPEAN
AVIATION

PRESS RELEASE

Inefficiency in the European air traffic management network resulting in an average additional fuel burn of 8.6%-11.2%

8 December 2020

THIS COVERS

- AVIATION SUSTAINABILITY
- NETWORK OPERATIONS
- OPTIMISED OPERATIONAL PERFORMANCE | N...

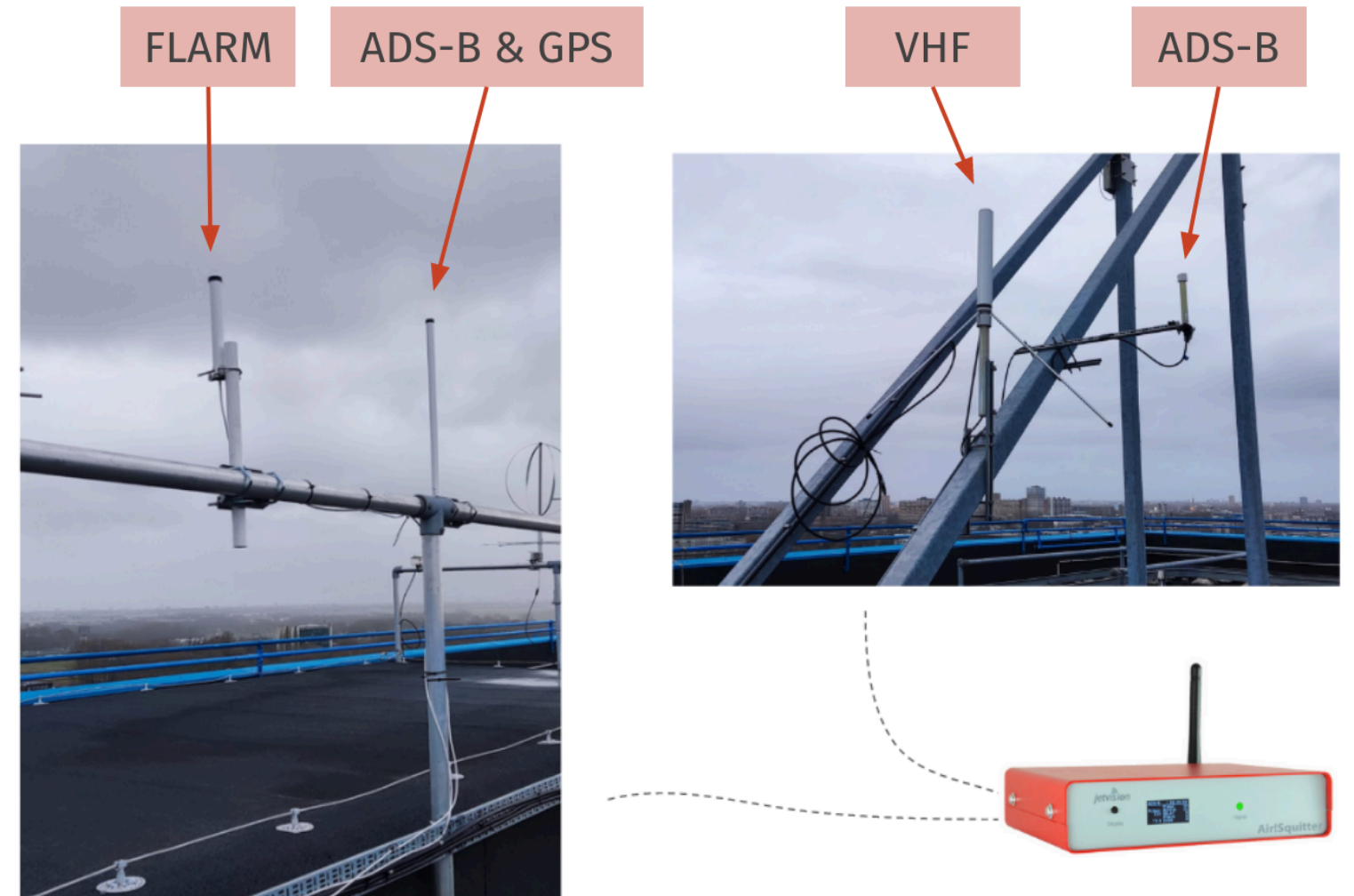


Inefficiency in the European air traffic management network (ATM) means that flights in Europe are using on average between **8.6% and 11.2% more fuel** than the most efficient flights, according to a new report issued today by EUROCONTROL.

Extra fuel burn from operational inefficiencies

The research context

- **Open data infrastructure:** aircraft surveillance, OpenSky, pyModeS, pyopensky, FastMeteo.
- **Performance and AI models:** OpenAP, trajectory prediction, weather reconstruction, contrail detection.
- **Flight optimization:** trajectory generation, fuel and climate objectives, operational constraints.
- **Sustainability impact:** emissions, contrails, capacity, and decision support for lower-impact operations.



ADS-B, Mode S, and VHF receiver setup on the AE building roof

Overview of this lecture

1. **Open flight data:** turn surveillance data into cleaned trajectories for climate analysis.
2. **Emissions:** use aircraft performance models to estimate fuel burn and pollutants.
3. **Weather and contrails:** connect trajectories to meteorology and persistent-contrail conditions.
4. **Optimization:** compare fuel-optimal, contrail-aware, and operationally feasible trajectories.

Objective: understand how open aviation data, performance models, weather data, and optimization can be combined into a reproducible workflow for reducing aviation climate impact.

The question that started this line of research

Is it possible to use only open data to study aircraft performance, simulate trajectories, infer fuel burn, and estimate emissions?

- This was the core research question behind OpenAP.
- The answer became a research stack: OpenSky, pyopensky, OpenAP, FastMeteo, contrail models, and opentop.
- Today's lecture is about how that stack becomes a climate-impact workflow.

Why openness matters in aviation climate research

- Aviation climate studies combine many uncertain assumptions.
- Proprietary data and closed models make those assumptions hard to inspect.
- Open workflows let others rerun, audit, challenge, and extend the result.
- This is especially important when outputs support operational or policy decisions.

Data

[OpenSky](#), ADS-B, Mode S, ADS-C

Models

[OpenAP](#), [FastMeteo](#), contrail physics

Optimization

[opentop](#), cost grids, trajectory constraints

Scrutiny

versioned code, open assumptions, reproducible figures

What is runnable in this lecture? CODE

The code slides use cached sample data so the lecture can render without OpenSky credentials or large weather downloads.

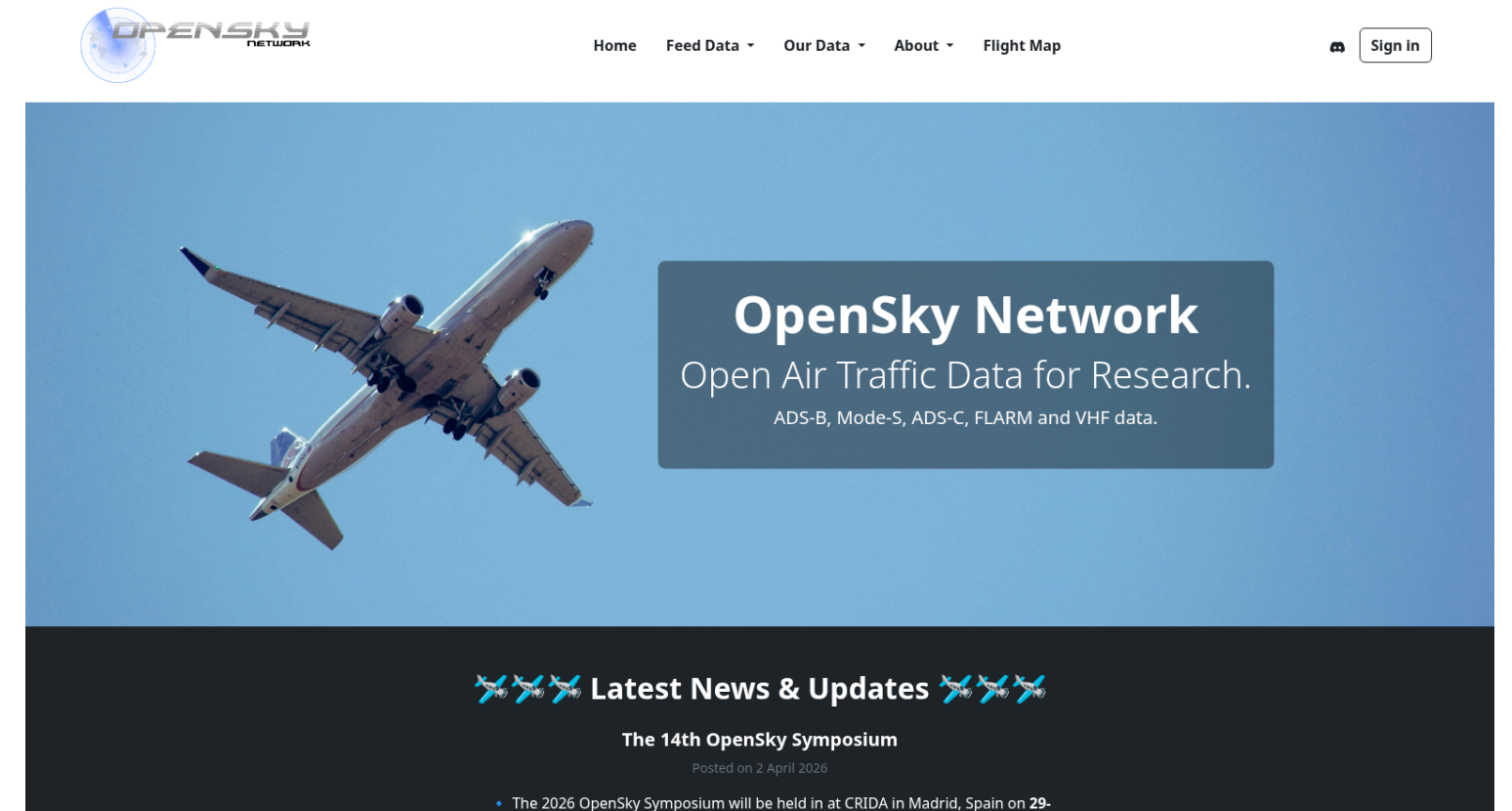
```
1 from isa_demo import load_sample_flight
2
3 flight = load_sample_flight()
4 flight[["timestamp", "typecode", "origin", "destination", "latitude", "longitude", "altitude"]].head()
```

	timestamp	typecode	origin	destination	latitude	longitude	altitude
0	2018-01-02 19:53:00+00:00	a319	LBG	BMA	49.085861	2.349666	8200.0
1	2018-01-02 19:54:00+00:00	a319	LBG	BMA	49.153427	2.310861	9475.0
2	2018-01-02 19:55:00+00:00	a319	LBG	BMA	49.233367	2.361121	9975.0
3	2018-01-02 19:56:00+00:00	a319	LBG	BMA	49.319916	2.418471	11225.0
4	2018-01-02 19:57:00+00:00	a319	LBG	BMA	49.411652	2.478896	14175.0

Open Flight Data

OpenSky is more than state vectors

- State vectors: positions, velocities, altitude, vertical rate.
- Raw Mode S: aircraft-derived parameters, richer surveillance messages.
- Flight lists: historical flight-level filtering and aggregation.
- Aircraft database: type and registration lookup for model assignment.
- ADS-C: remote and oceanic trajectory/intention information.



OpenSky Network homepage

Most climate workflows start with this table

Field	Climate-use role	Typical issue
time	weather merge, duration, traffic state	irregular sampling
icao24 , callsign	flight and aircraft identity	callsign ambiguity
latitude, longitude	trajectory, sector, route	coverage gaps
altitude	phase, contrail level, fuel model	barometric vs geometric altitude

pyopensky access patterns CODE

This cell is disabled for reliable rendering. It shows the intended live and historical query paths.

Live state vectors: REST

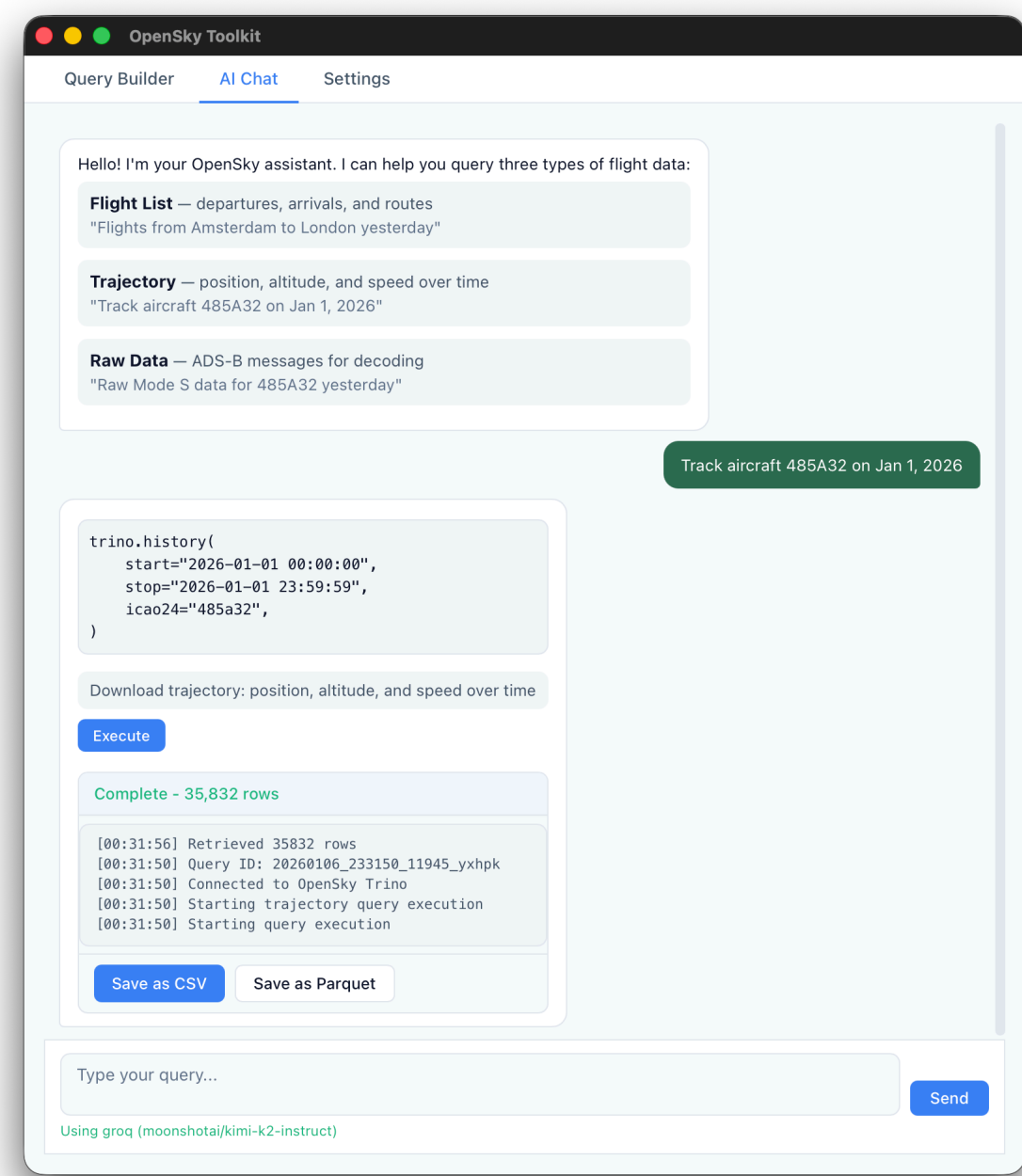
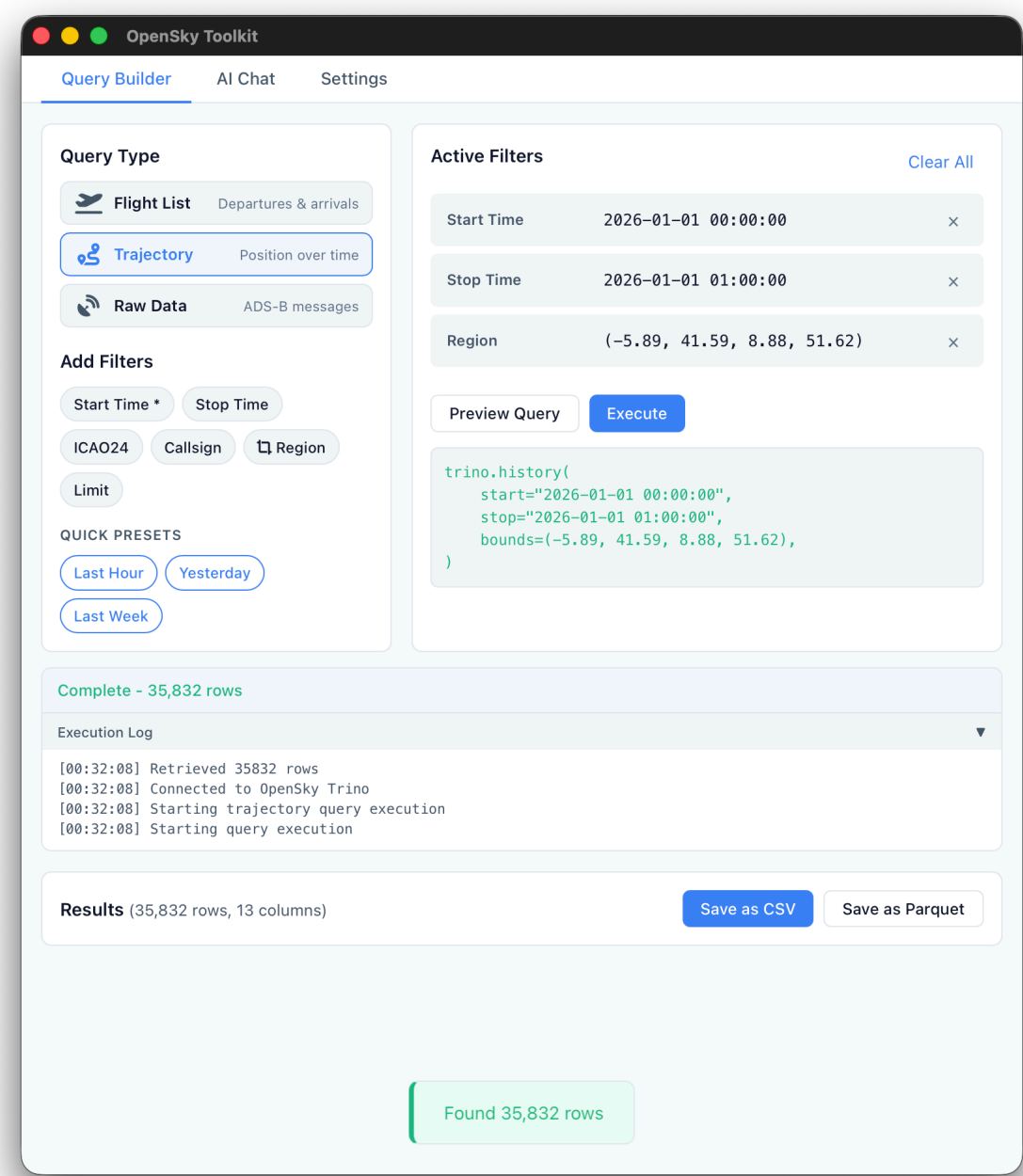
```
1 from pyopensky.rest import REST
2
3 rest = REST()
4 states = rest.states()
5 track = rest.tracks(icao24="3c6444")
```

Historical state vectors: Trino

```
1 from pyopensky.trino import Trino
2
3 trino = Trino()
4 flights = trino.flightlist(
5     start="2024-06-01",
6     stop="2024-06-02",
7     airport="EHAM",
8 )
9
10 states = trino.history(
11     start="2024-06-01 12:00",
12     stop="2024-06-01 13:00",
13     bounds=(3.0, 7.5, 50.0, 54.0),
14 )
```

OSTK: OpenSky Toolkit

OSTK is a desktop toolkit for OpenSky historical-data workflows: <https://github.com/junzis/ostk>



Load and inspect one trajectory CODE

Start with one cached OpenSky flight so the rest of the lecture can run without credentials or a live database query.

```
1 flight = load_sample_flight()  
2 flight.shape, flight.timestamp.min(), flight.timestamp.max()
```

```
((116, 15),  
 Timestamp('2018-01-02 19:53:00+0000', tz='UTC'),  
 Timestamp('2018-01-02 21:48:00+0000', tz='UTC'))
```

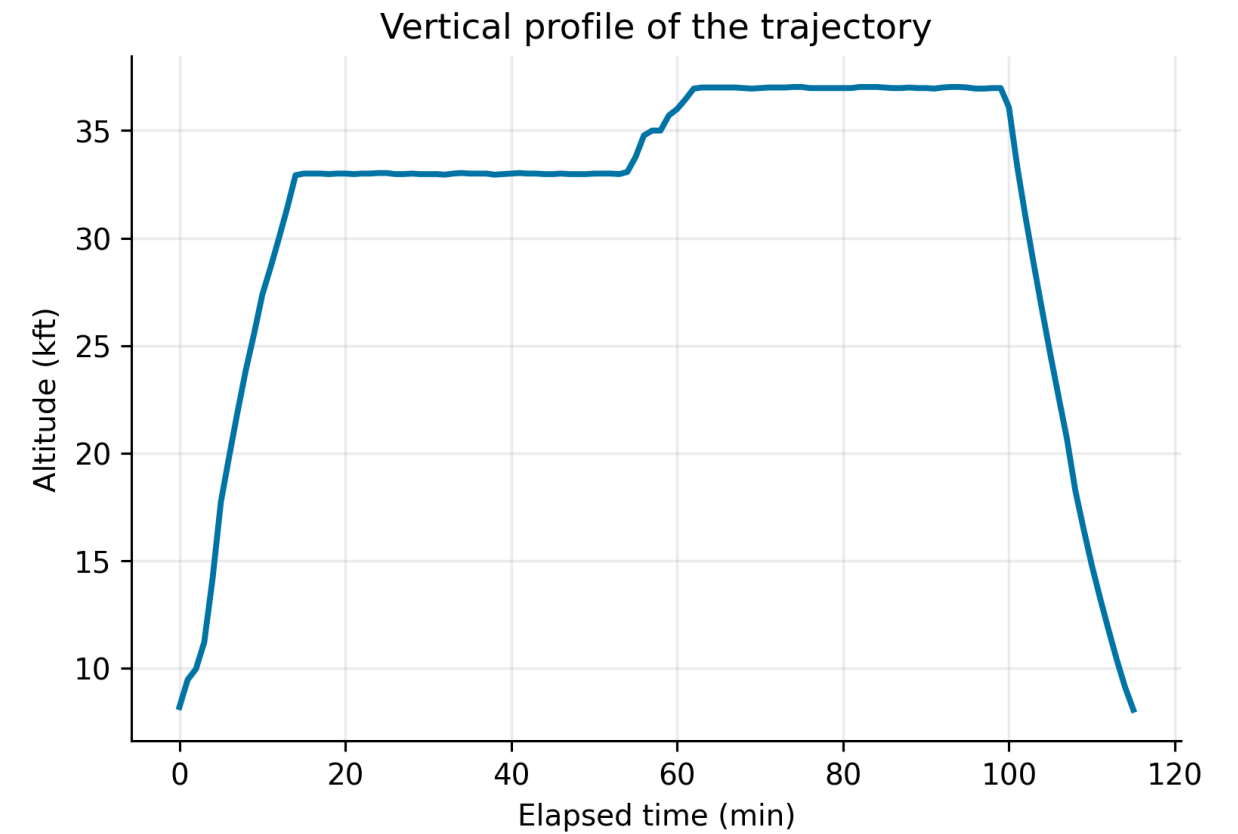
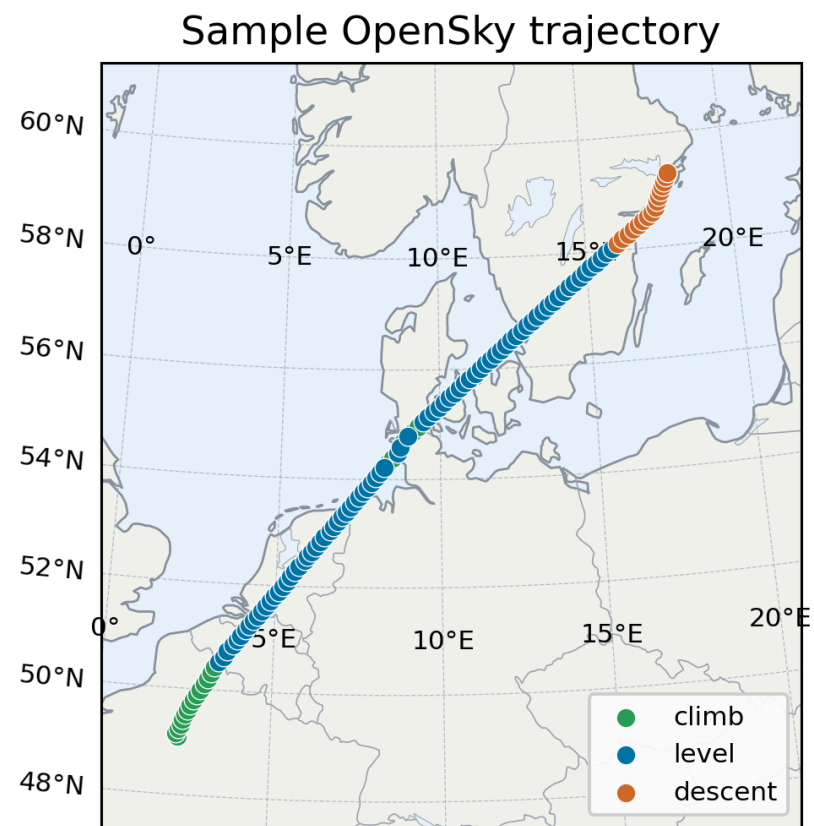
```
1 flight.phase.value_counts()
```

```
phase  
level      80  
climb      20  
descent    16  
Name: count, dtype: int64
```

Trajectory geometry and vertical behavior CODE

```
1 from isa_demo import plot_trajectory
2
3 plot_trajectory(flight)
```

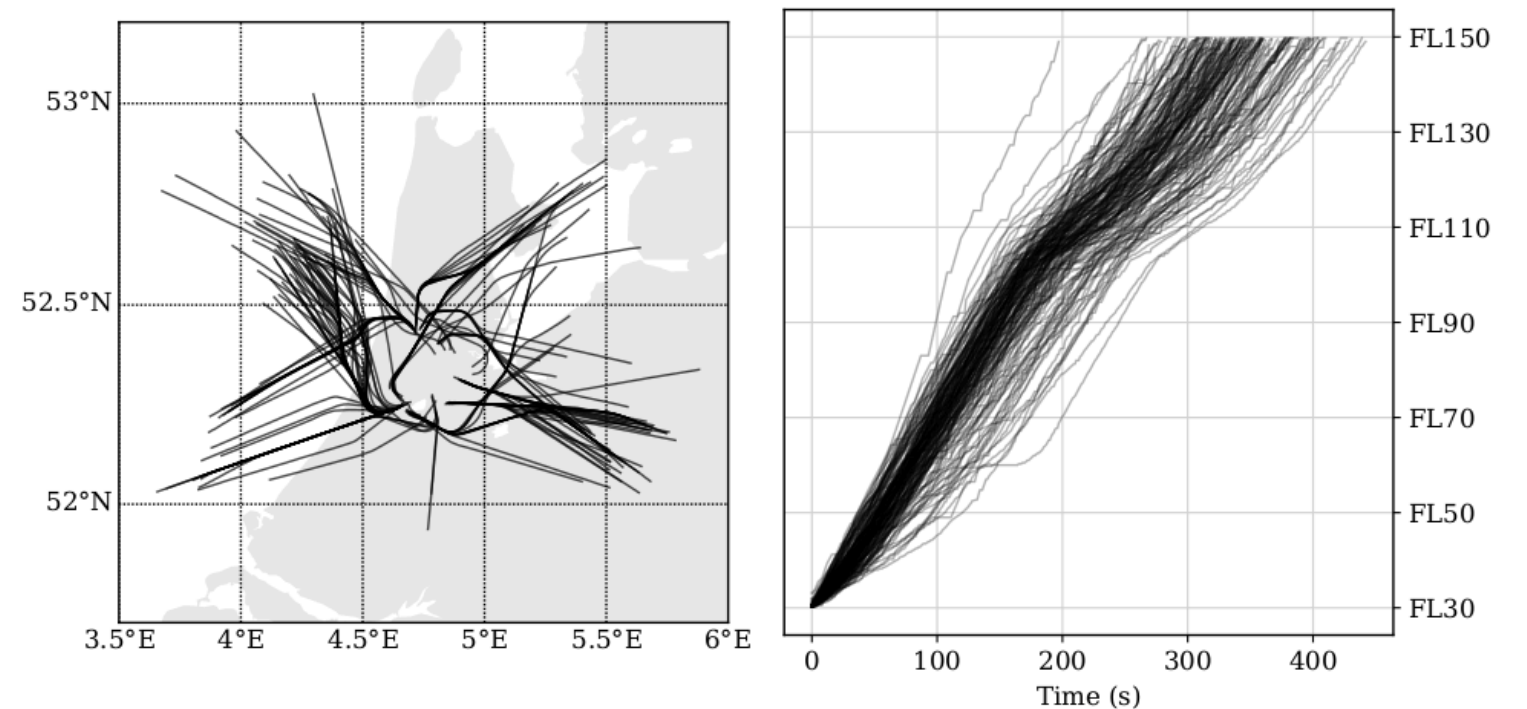
```
1 from isa_demo import plot_vertical_profile
2
3 plot_vertical_profile(flight)
```



From Trajectories to Emissions

Why OpenAP exists

- Surveillance tells us what happened.
- Performance models estimate the physical effort required.
- Fuel and emissions need aircraft type, engine, mass, speed, altitude, and atmosphere.
- OpenAP was built to make that bridge open and reproducible.



Open data behind performance modeling

OpenAP model stack

Kinematic

Speed, altitude, and vertical-rate envelopes from open trajectories

Dynamic

Drag polar, thrust, fuel flow, aircraft mass sensitivity

Environment

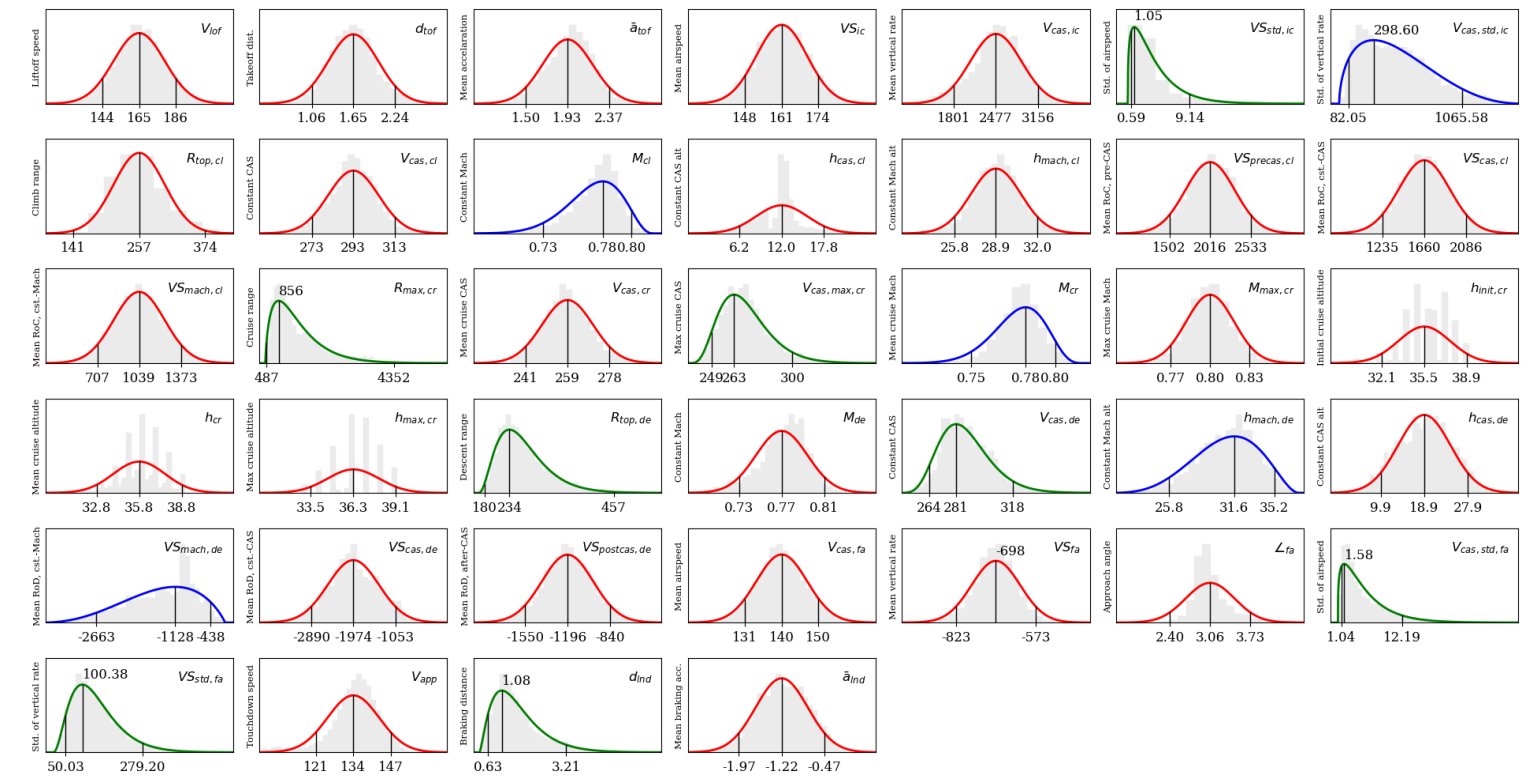
OpenAP emission indices; FastMeteo for wind, temperature, and humidity

Optimization

Trajectory objectives, constraints, and climate-aware routing

Kinematics: learning flight behavior from data

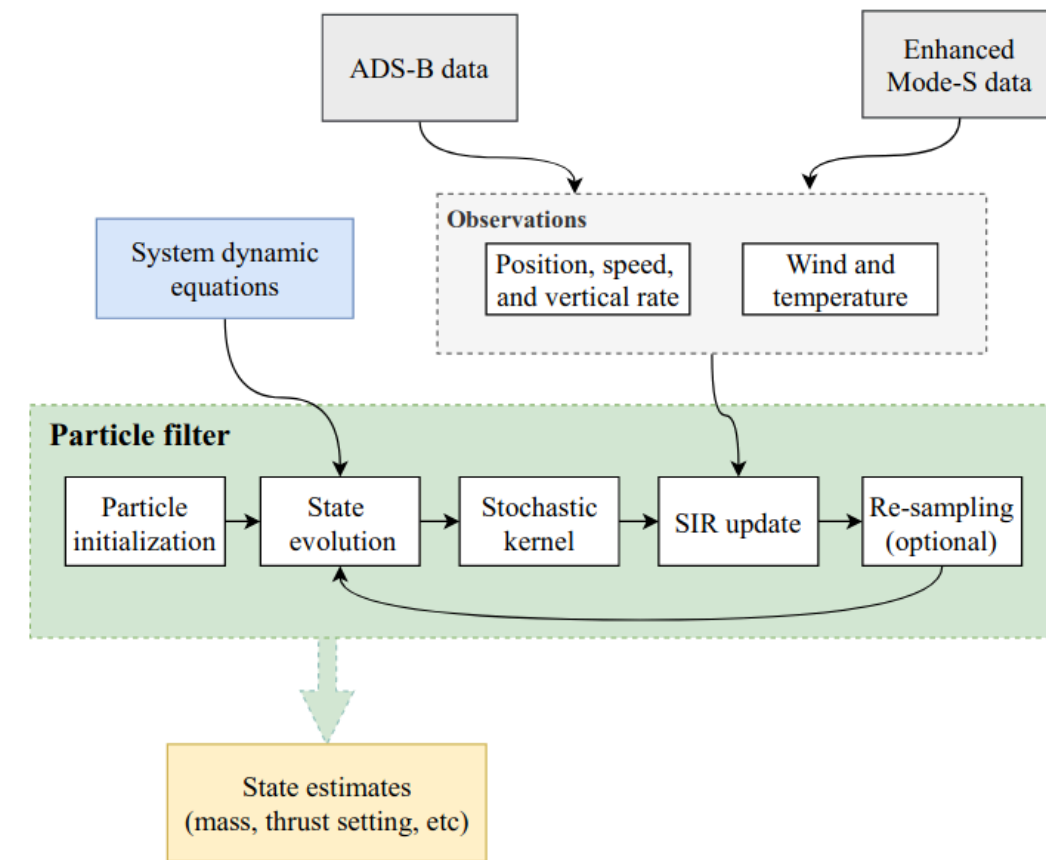
- Open trajectories reveal common speed and vertical-rate profiles.
- These models support simulation and trajectory generation.
- They also expose where the observed trajectory is operationally unusual.



OpenAP kinematic profiles

Dynamic models: drag, thrust, and mass

- Dynamic models connect flight state to drag, thrust, fuel flow, and mass.
- Drag-polar coefficients can be estimated from open ADS-B trajectories.
- Mass is the hardest hidden state: it drives climb, fuel burn, and prediction.
- Particle filters combine ADS-B climb-rate evolution with OpenAP dynamics.
- Recent work uses the EUROCONTROL PRC Data Challenge 2024 dataset.



Particle filter for mass estimation during climb

Aircraft model inputs for mass and emissions CODE

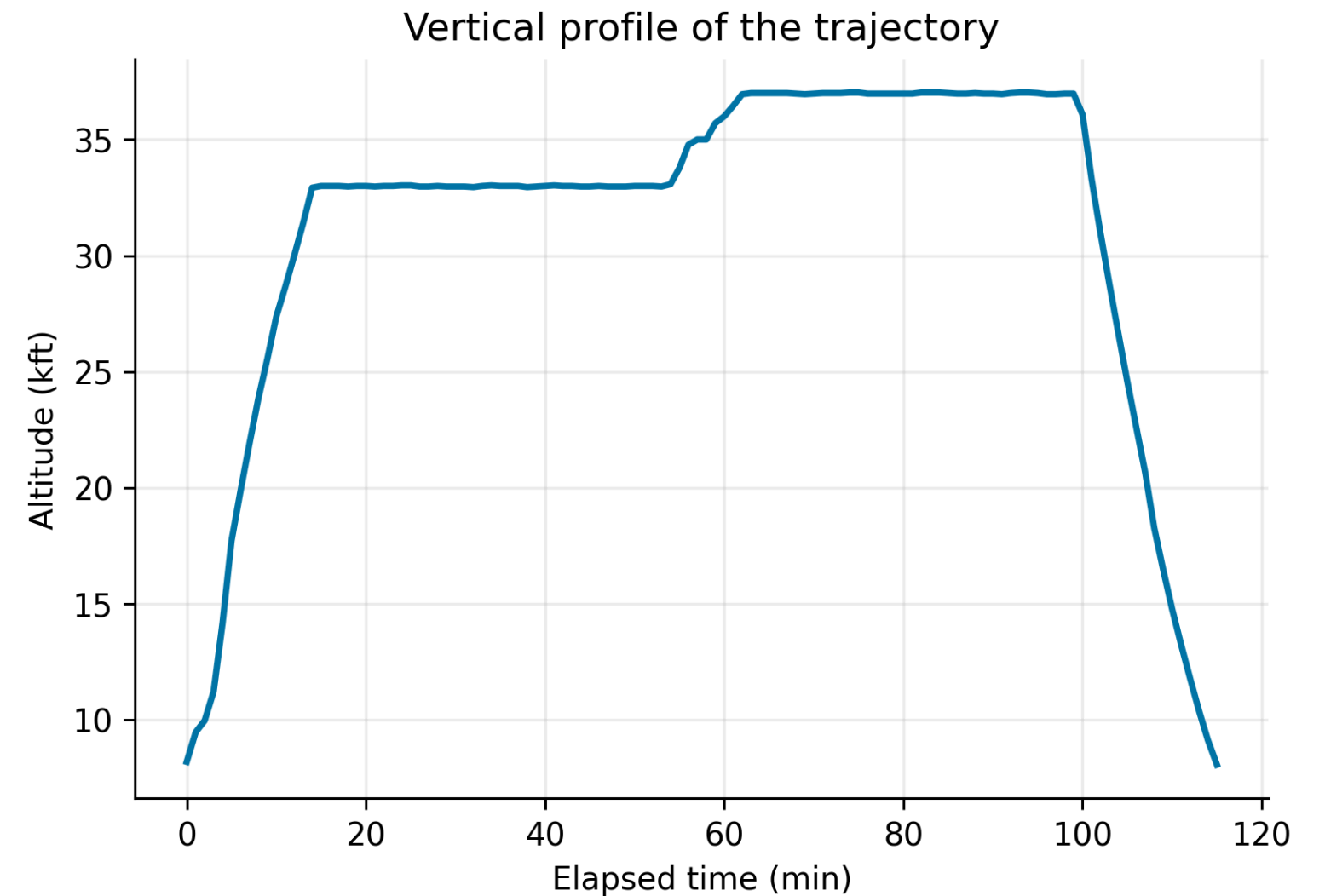
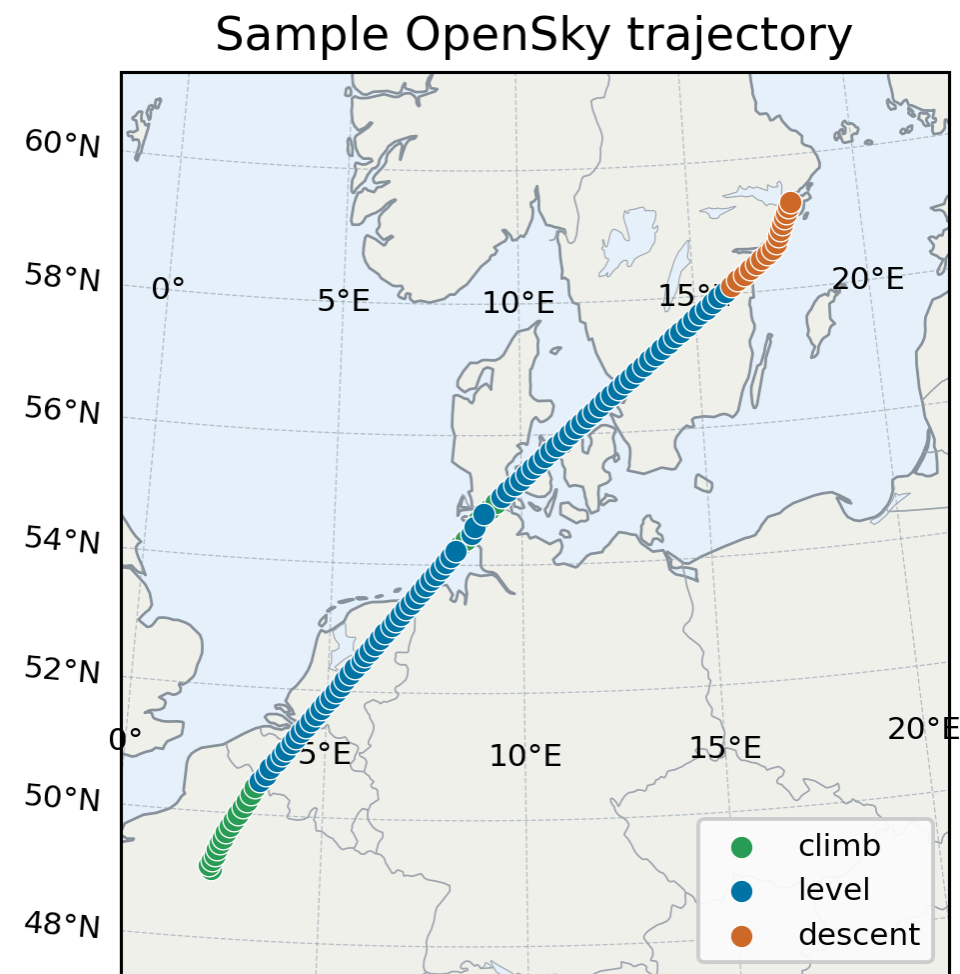
OpenAP exposes the aircraft limits and default engine used by the performance model; these values define the mass range behind fuel, emissions, and trajectory predictions.

```
1 import openap
2
3 aircraft = openap.prop.aircraft("A319")
4 aircraft
```

```
{'aircraft': 'Airbus A319',
 'mtow': 75500,
 'mlw': 62500,
 'oew': 40800,
 'mfc': 30190,
 'vmo': 350,
 'mmo': 0.82,
 'ceiling': 12500,
 'pax': {'max': 156, 'low': 110, 'high': 140},
 'fuselage': {'length': 33.84, 'height': 4.14, 'width': 3.95},
 'wing': {'area': 124, 'span': 35.8, 'mac': 4.1935, 'sweep': 25, 't/c': None},
 'flaps': {'type': 'single-slotted',
 'area': 21.1,
 'bf/b': 0.78,
 'lambda_f': 0.9,
 'cf/c': 0.176,
 'Sf/S': 0.17},
```

Reminder: the example flight

Before estimating fuel and emissions, return to the same A319 trajectory used in the data section.



Fuel and emissions from a trajectory CODE

```
1 import pandas as pd
2 import openap
3
4 fuelflow = openap.FuelFlow("A319")
5 emission = openap.Emission("A319")
6
7 mass = 62_000.0
8 rows = []
9
10 for point in flight.itertuples(index=False):
11     ff = float(
12         fuelflow.enroute(
13             mass=mass,
14             tas=point.airspeed,
15             alt=point.altitude,
16             vs=float(point.vertical_rate),
17         )
```

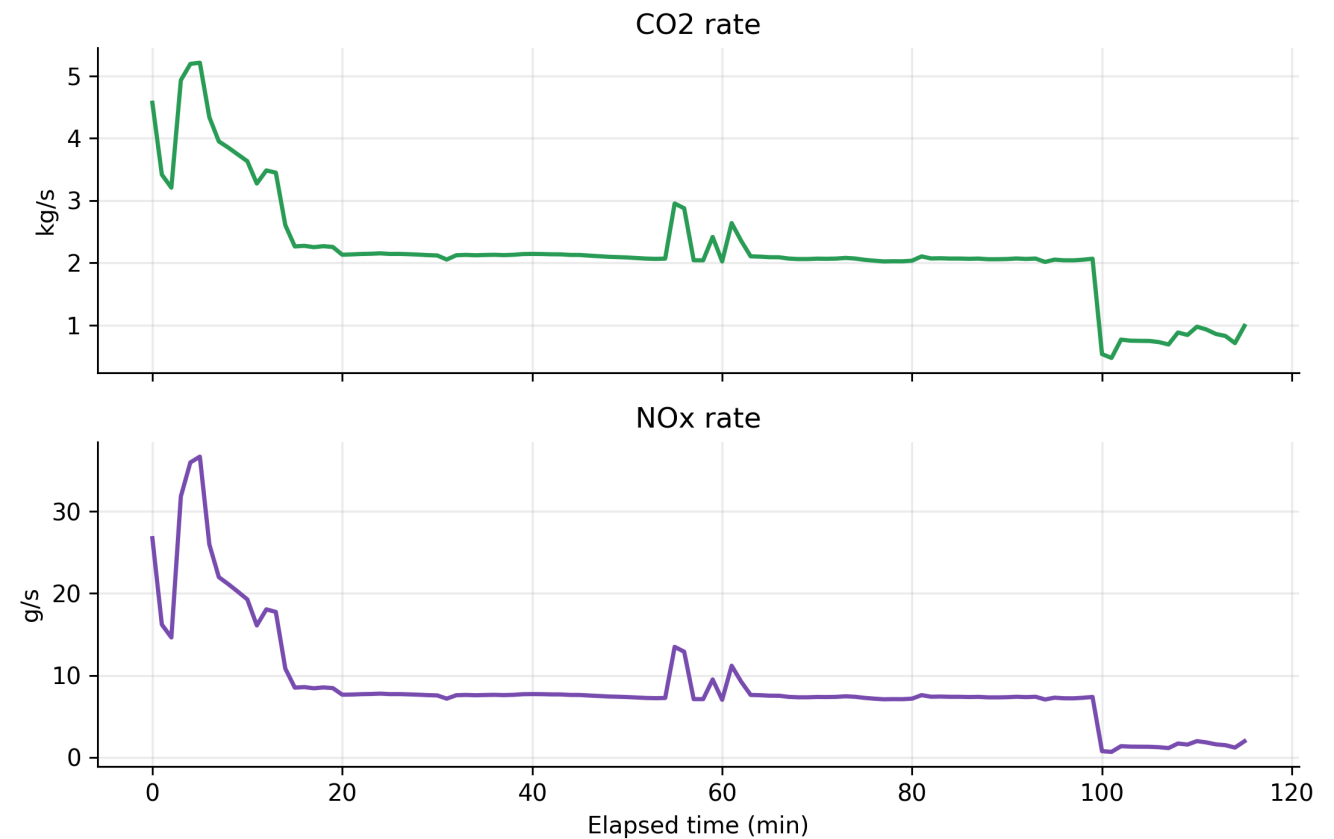
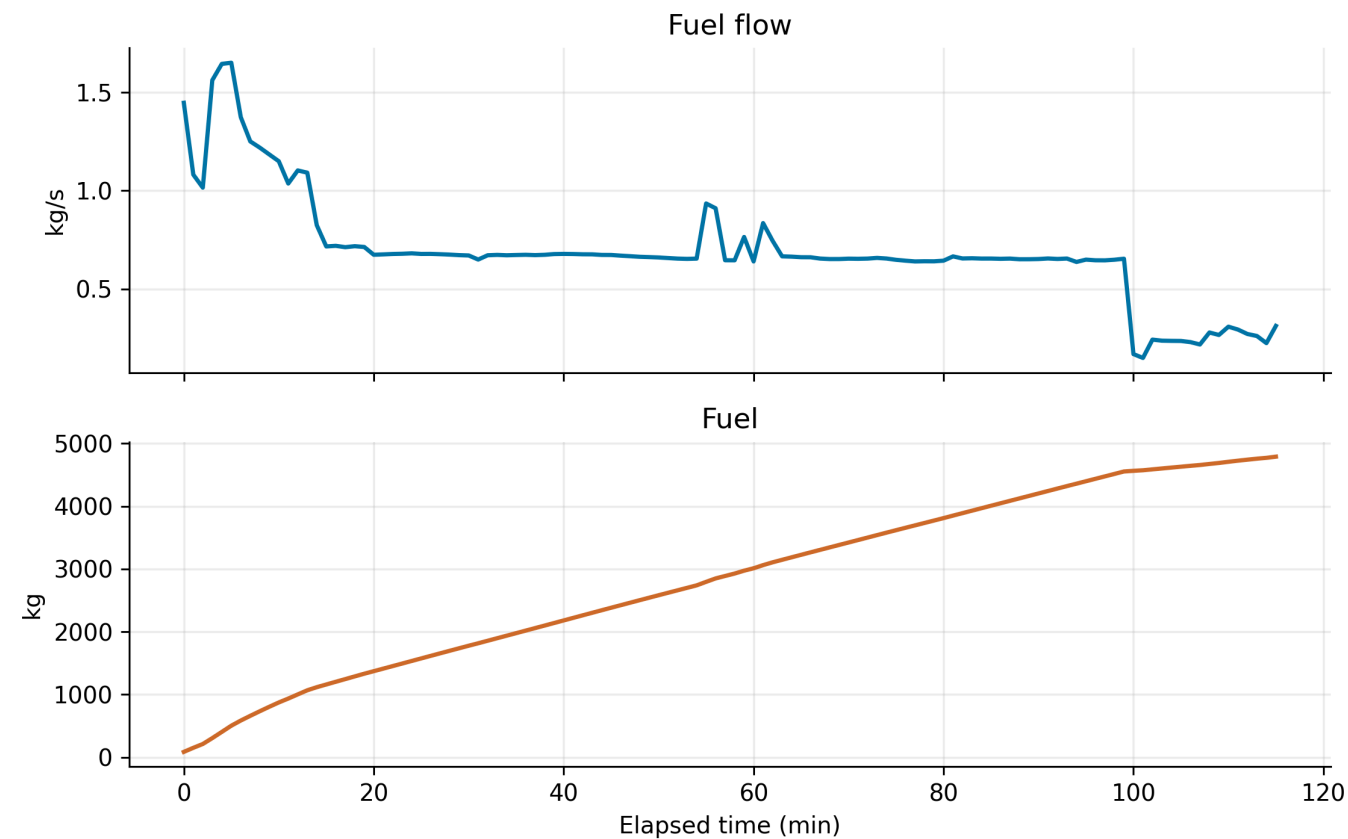
	elapsed_min	altitude	groundspeed	fuel_flow_kg_s	fuel_kg	co2_kg	nox_g
0	0.0	8200.0	255.0	1.445910	86.754630	274.144630	1604.411907
1	1.0	9475.0	290.0	1.080693	64.841556	204.899317	971.669363
2	2.0	9975.0	332.0	1.014561	60.873660	192.360765	877.380513
3	3.0	11225.0	353.0	1.560502	93.630148	295.871266	1910.257630
4	4.0	14175.0	368.0	1.643420	98.605174	311.592351	2158.986260

Emission estimate along the flight CODE

```
1 totals = flight_perf[["fuel_kg", "co2_kg", "nox_g"]].sum()
2 totals.round(1)
```

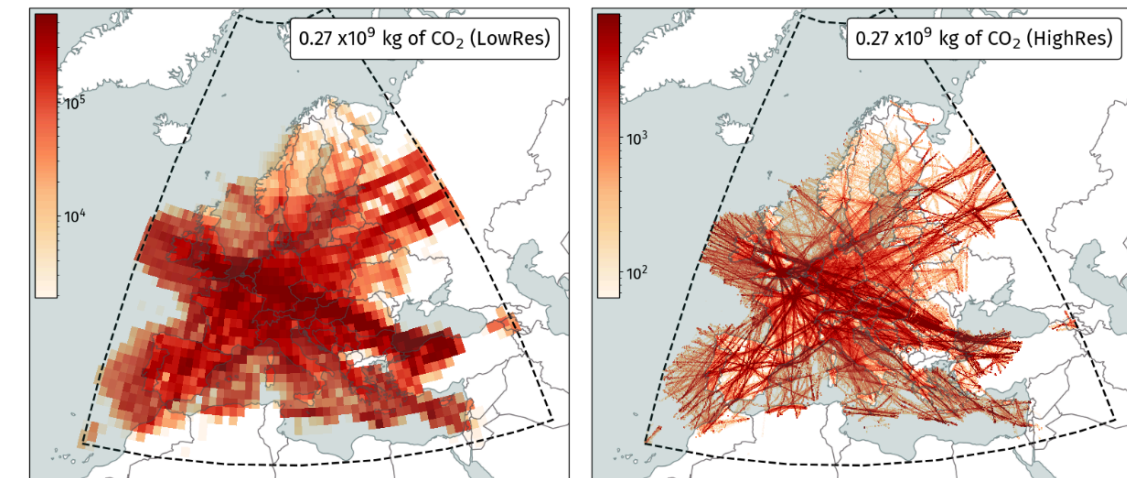
```
fuel_kg      4789.0
co2_kg       15133.4
nox_g        60650.1
dtype: float64
```

```
1 from isa_demo import plot_emission_timeseries
2
3 plot_emission_timeseries(flight_perf)
```

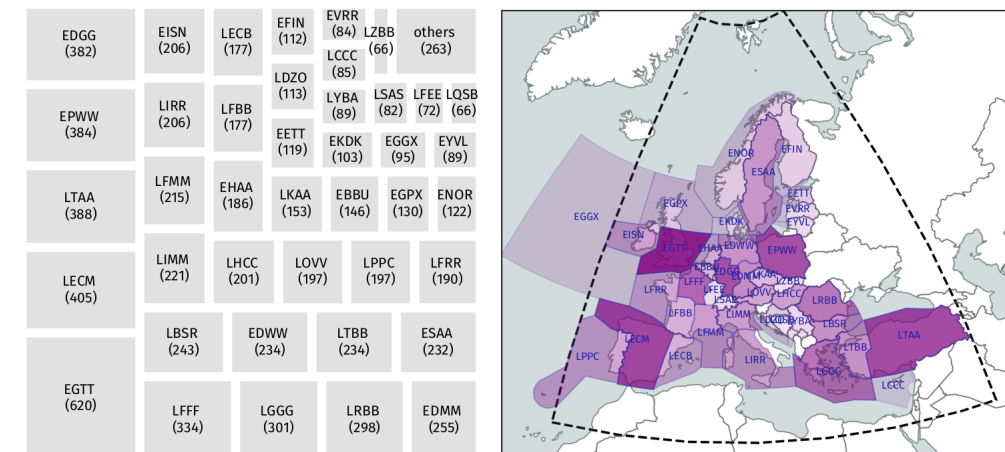


Bottom-up emission estimation

- Start from individual trajectories, not schedules or great-circle routes.
- Estimate fuel and emissions for every flight segment.
- Aggregate by grid cell, airline, FIR, aircraft type, or flight level.
- More realistic than schedule + great-circle approximations.
- Captures route, altitude, and holding/inefficiency effects.
- **Limitation:** surveillance coverage, mass assumptions, and model accuracy.



Open-data-driven CO₂ over Europe

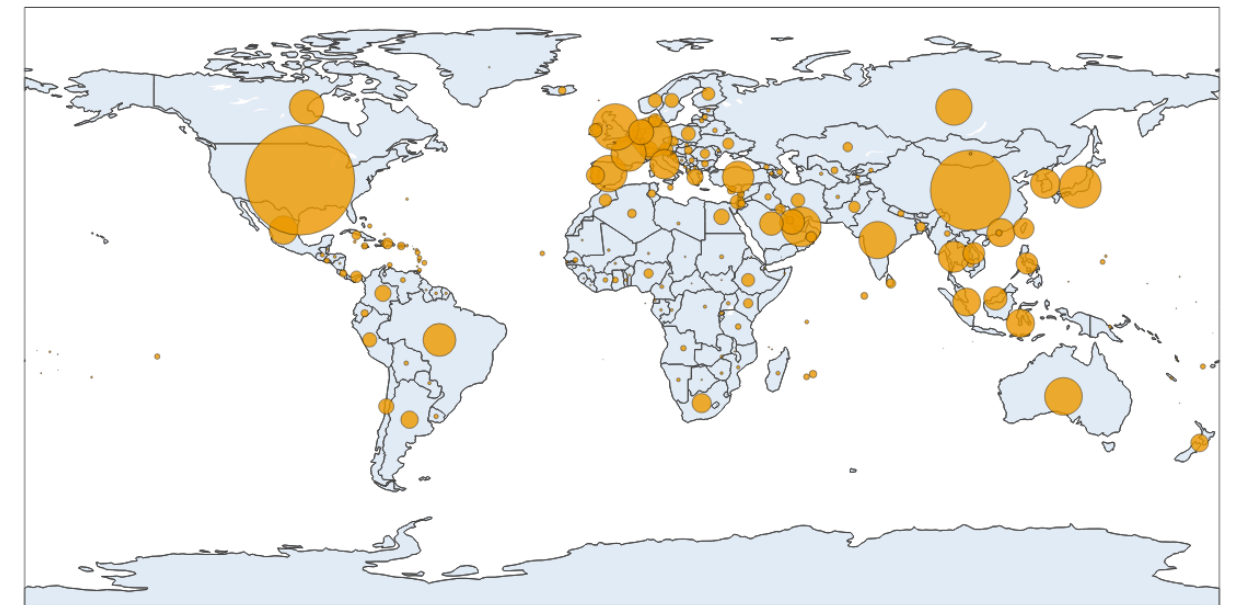


FIR-level CO₂ aggregation

AeroSCOPE: using open data to tackle emissions coverage gaps

- AeroSCOPE combines open traffic sources into a global route and emissions database.
- It addresses data gaps with estimation methods where regional datasets are missing.
- Population and econometric proxies can estimate flows in regions with weak surveillance coverage, such as Africa, South America, and parts of Asia.
- Dataset and code:
https://github.com/AeroMAPS/AeroSCOPE_dataset

Country values for CO2 (kg)

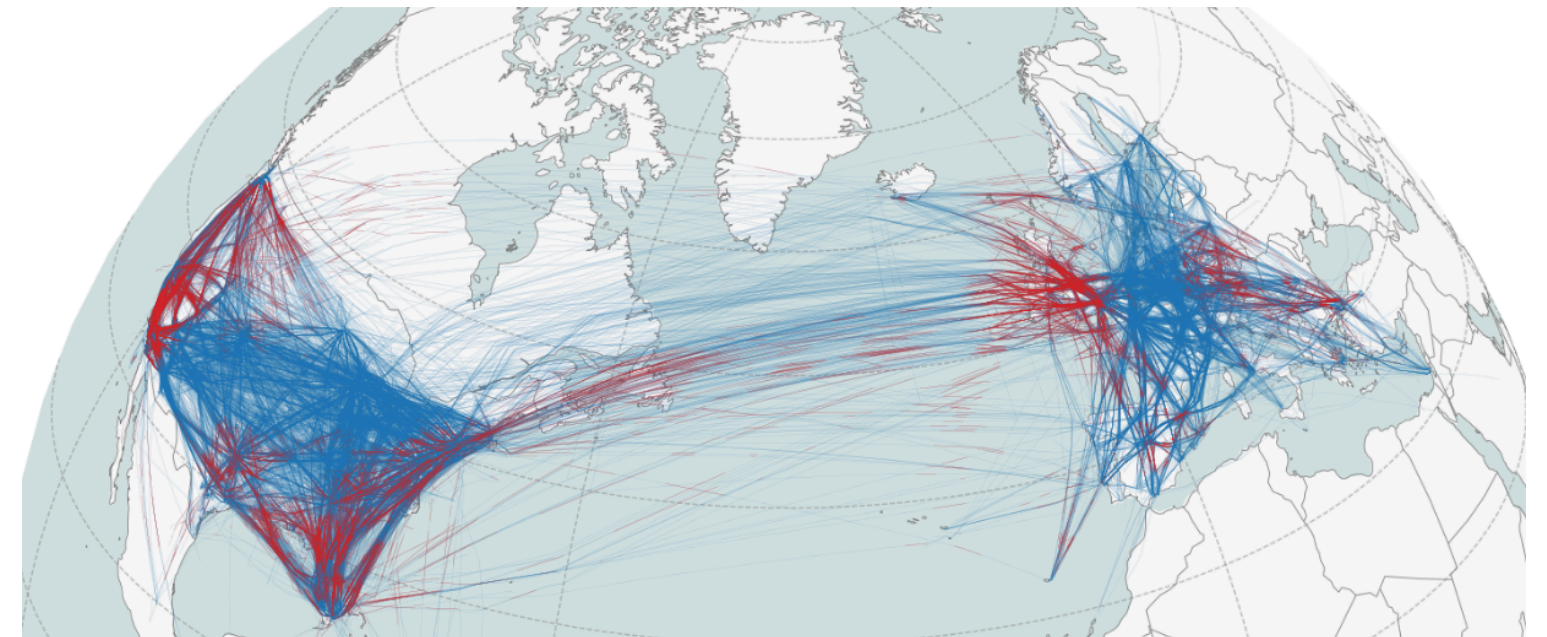


AeroSCOPE map

Contrails and Weather

Contrails are where the open-data workflow becomes fragile

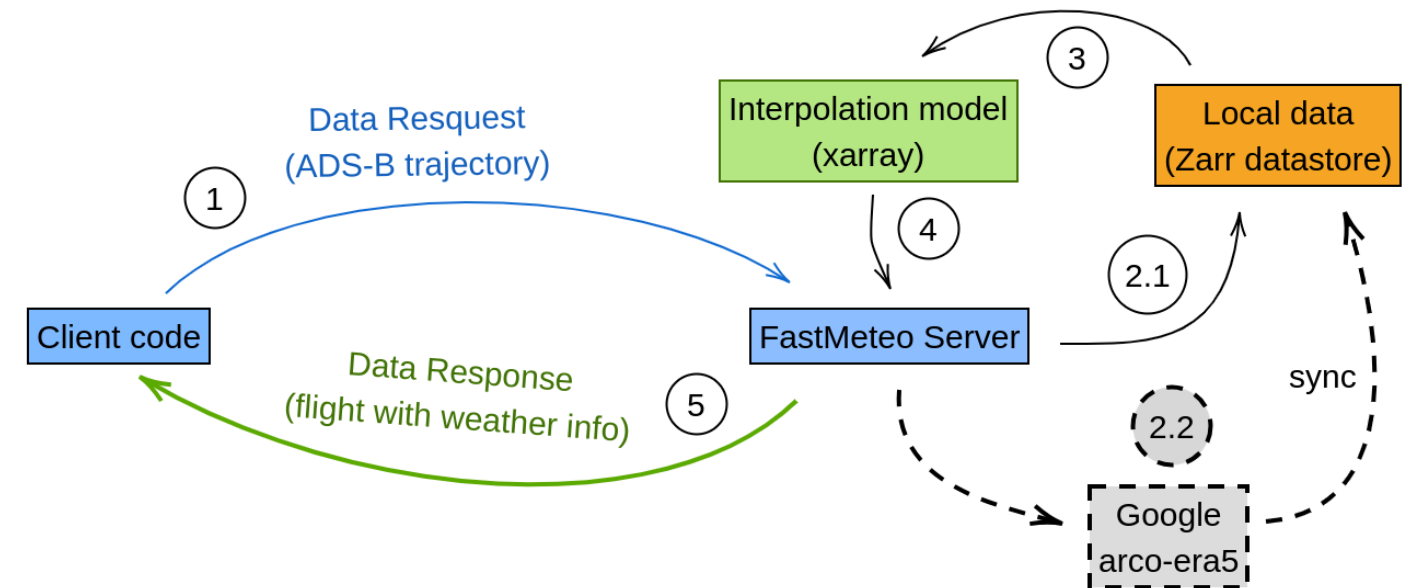
- CO2 is tied directly to fuel burn.
- Contrails depend on atmospheric state, altitude, time, and persistence.
- A small subset of flights can dominate short-term warming.
- Forecast uncertainty can decide whether a rerouting is beneficial or harmful.



Persistent contrail formation from OpenSky Report

Weather data bottlenecks led to FastMeteo

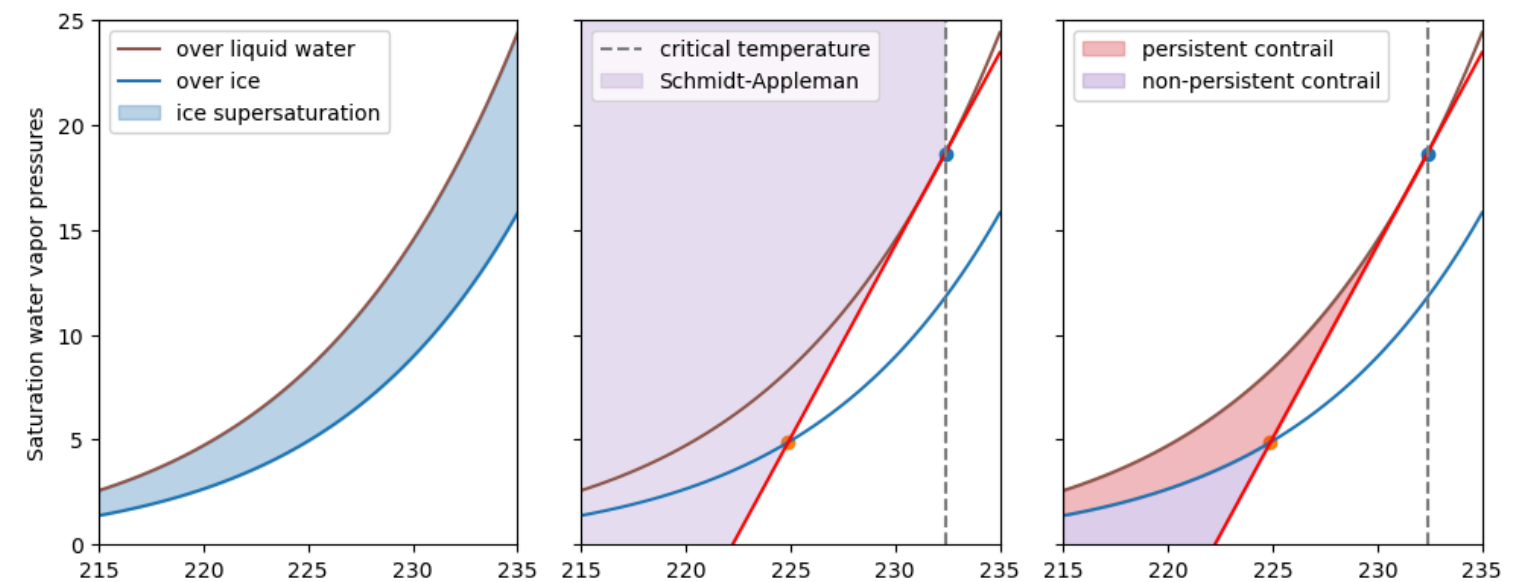
- Contrail prediction needs weather sampled along each trajectory.
- Traditional workflows move large GRIB/NetCDF files to extract small slices.
- For millions of flights, this becomes the computational bottleneck.
- FastMeteo interpolates ERA5 and forecast fields at trajectory points.
- Useful for wind-aware fuel estimation and contrail screening.



FastMeteo server/client architecture

What parameters do contrail methods need?

- Pressure or altitude from the trajectory.
- Temperature from meteorological data.
- Humidity from meteorological data.
- Formation condition: Schmidt-Appleman criterion.
- Persistence condition: ice supersaturation.



Contrail calculation curves

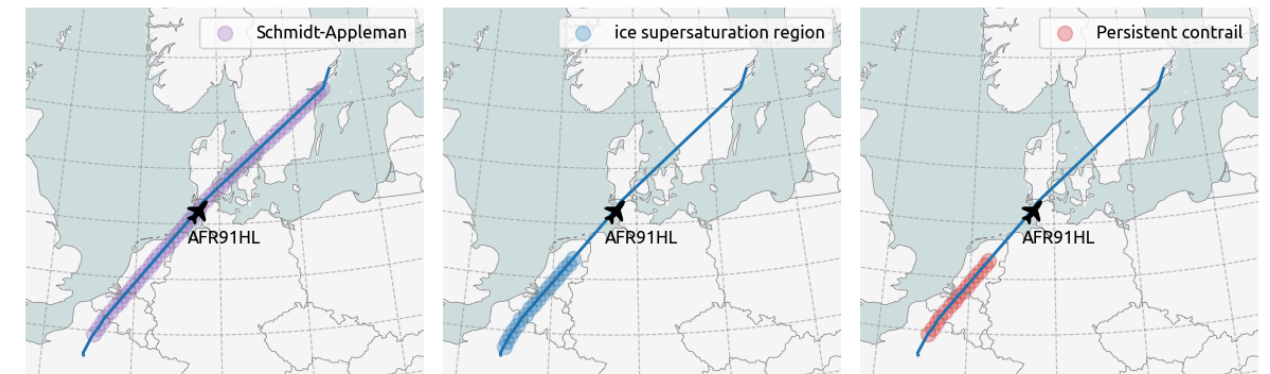
Add real weather with FastMeteo CODE

Use the example flight from the FastMeteo paper workflow. The paper notebook used [Grid](#); the current API uses [ArcoEra5](#).

```
1 import pandas as pd
2 from fastmeteo.source import ArcoEra5
3
4 flight = pd.read_csv(
5     "assets/fastmeteo_example_flight.csv",
6     parse_dates=["timestamp"],
7 )
8
9 era5 = ArcoEra5(local_store="/tmp/era5-zarr")
10 flight_weather = era5.interpolate(flight)
```

Diagnose contrail regions with OpenAP CODE

```
1 from openap import aero, contrail
2
3 flight_contrail = flight_weather.assign(
4     rhi=lambda d: contrail.relative_humidity(
5         d.specific_humidity,
6         aero.pressure(d.altitude * aero.ft),
7         d.temperature,
8         to="ice",
9     ),
10    crit_temp=lambda d: contrail.critical_temperature_water(
11        aero.pressure(d.altitude * aero.ft)
12    ),
13    sac=lambda d: d.temperature < d.crit_temp,
14    issr=lambda d: d.rhi > 1,
15    persistent=lambda d: d.sac & d.issr,
16 )
```

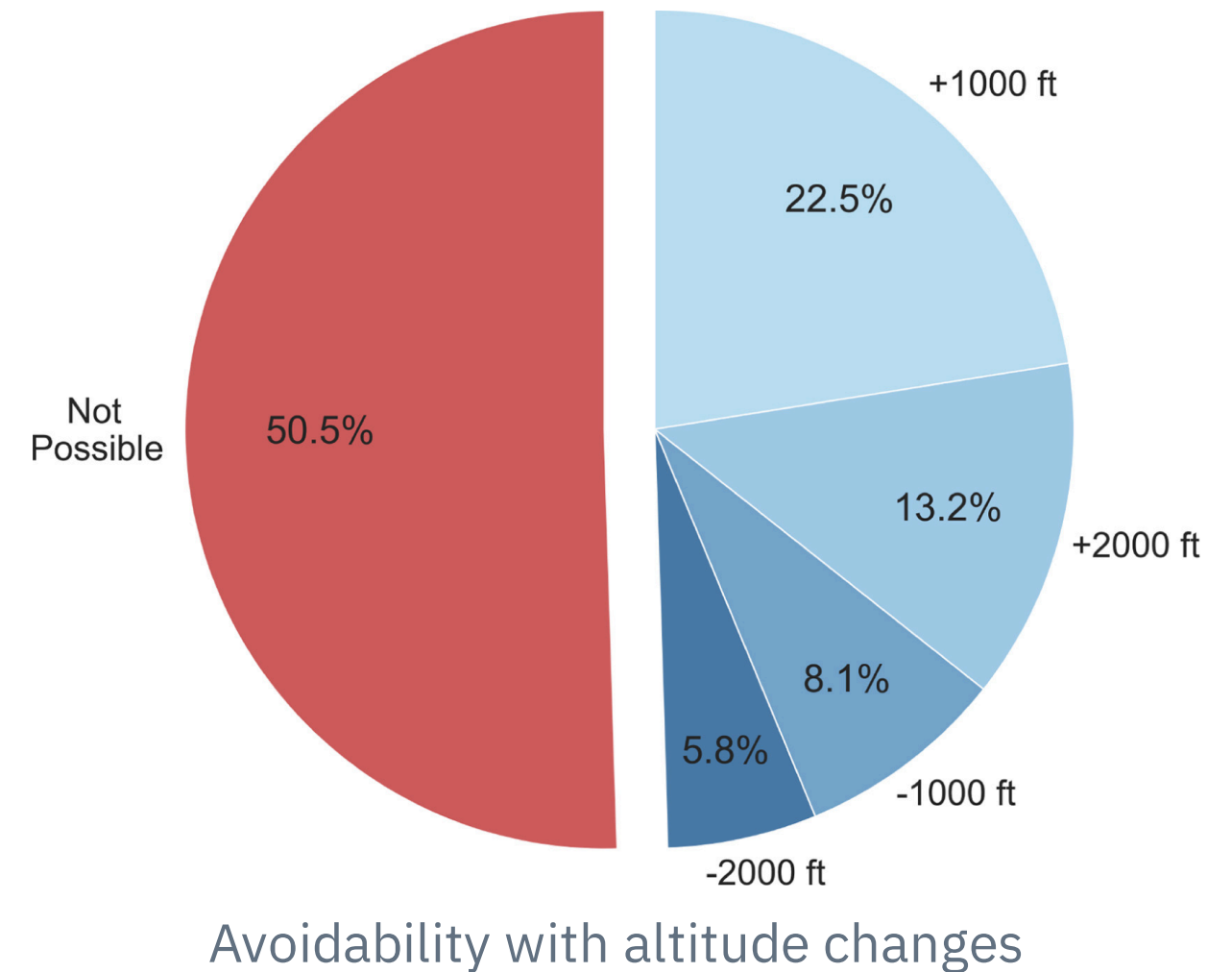


FastMeteo example: persistent contrail segments

Altitude changes can help

- Contrail-diversion analyses suggest large mitigation potential.
- Around half of persistent contrails can be avoided with altitude changes in a simplified setup.
- The hard question is whether the idea survives operational constraints.
- Paper: **Contrail minimization through altitude diversions: A feasibility study leveraging global data**

<https://doi.org/10.1016/j.trip.2023.100953>



Climate-Optimal Routing

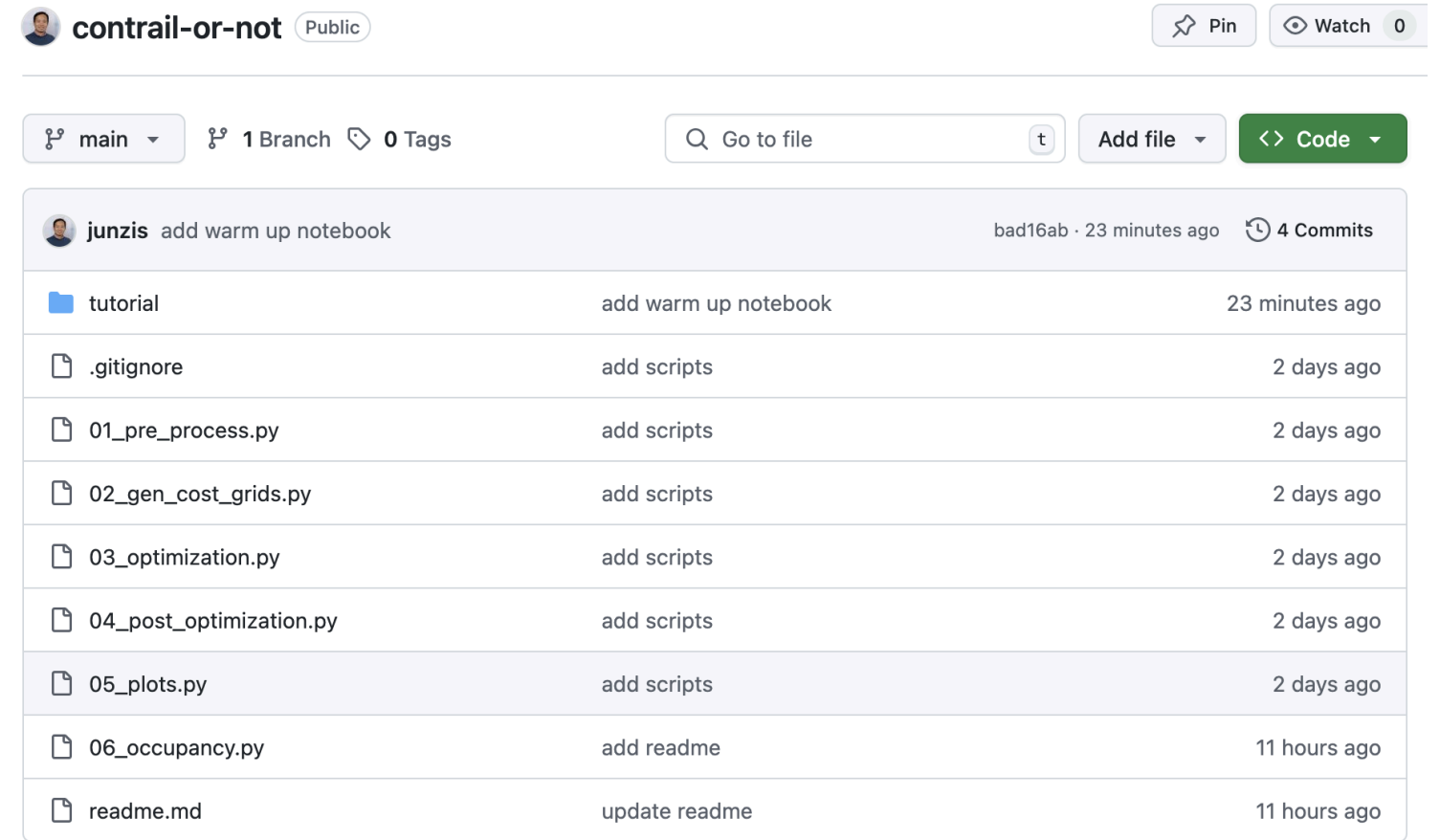
What was ignored in simple altitude-diversion studies?

- Altitude diversion shows mitigation potential, but it is not yet operational routing.
- A flight-plan change also changes speed, lateral path, wind exposure, fuel burn, and sector demand.
- This leads to the next feasibility question: can climate-optimal routing work as an ATM procedure?
- Paper: **Contrail, or not contrail, that is the question: the “feasibility” of climate-optimal routing**
<https://arxiv.org/abs/2504.13907>

This is the point where “contrail mitigation potential” becomes a routing, network, and governance problem.

The climate-optimal routing paper asks four questions

- **Fuel:** can we reduce contrails with reasonable excess fuel?
- **Weather:** can forecasts support rerouting decisions?
- **Network:** are contrail-optimized trajectories sustainable for air-traffic-flow management?
- **Governance:** who is responsible for climate-optimal routing?
- **Open workflow:**
<https://github.com/junzis/contrail-or-not>



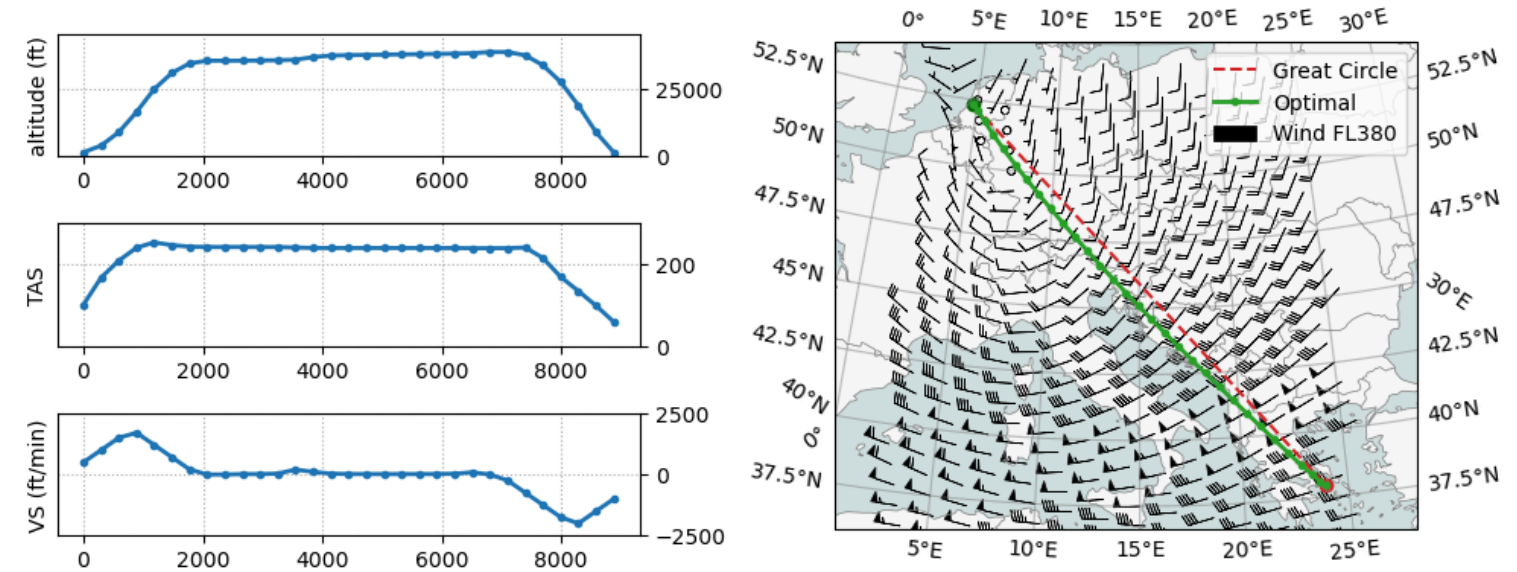
The screenshot shows the GitHub interface for the repository 'contrail-or-not'. At the top, the repository name and 'Public' status are visible, along with 'Pin' and 'Watch 0' buttons. Below this, the 'main' branch is selected, showing '1 Branch' and '0 Tags'. A search bar 'Go to file' and buttons 'Add file' and 'Code' are present. The commit history table shows a recent commit by 'junzis' titled 'add warm up notebook' with commit hash 'bad16ab' and '23 minutes ago', containing 4 commits. The file list includes a 'tutorial' folder and several Python scripts (.py) and a README file, each with a description of the last change and the time since the change.

File	Commit Message	Time
tutorial	add warm up notebook	23 minutes ago
.gitignore	add scripts	2 days ago
01_pre_process.py	add scripts	2 days ago
02_gen_cost_grids.py	add scripts	2 days ago
03_optimization.py	add scripts	2 days ago
04_post_optimization.py	add scripts	2 days ago
05_plots.py	add scripts	2 days ago
06_occupancy.py	add readme	11 hours ago
readme.md	update readme	11 hours ago

contrail-or-not repository

From feasibility questions to 4D trajectory generation

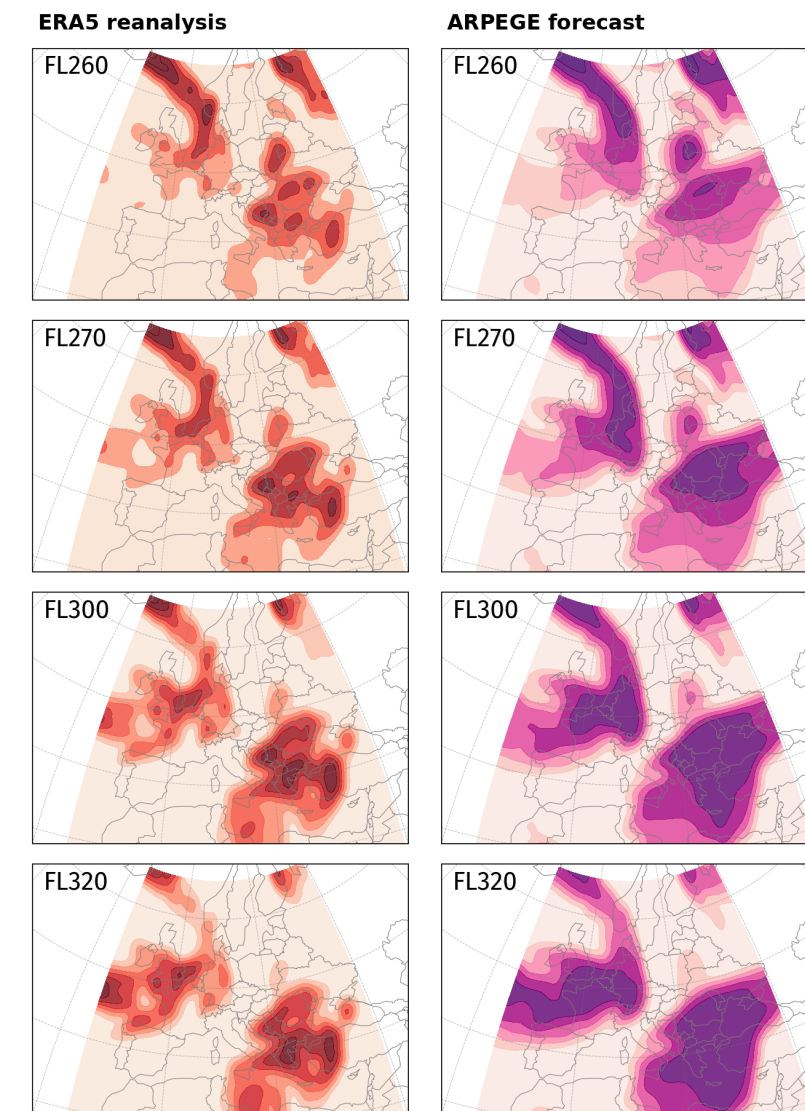
- To answer the feasibility questions, we need complete trajectories, not altitude tweaks on fixed routes.
- **opentop** optimizes altitude, speed, timing, and lateral path together.
- Wind, aircraft performance, and gridded atmospheric costs enter the same nonlinear optimal-control problem.
- This creates comparable fuel-optimal and climate-aware trajectories.



TOP trajectory optimization with wind

Cost grids turn weather into optimization input

- Contrail-forming regions become gridded costs.
- The optimizer samples the grid along candidate trajectories.
- Fuel and contrail terms must be balanced explicitly.
- Smooth interpolation reduces discontinuities in the optimization problem.
- **ERA5 reanalysis** versus **ARPEGE forecast** on 20 February 2022.



ERA5 and ARPEGE contrail cost grids

Climate objective in opentop CODE

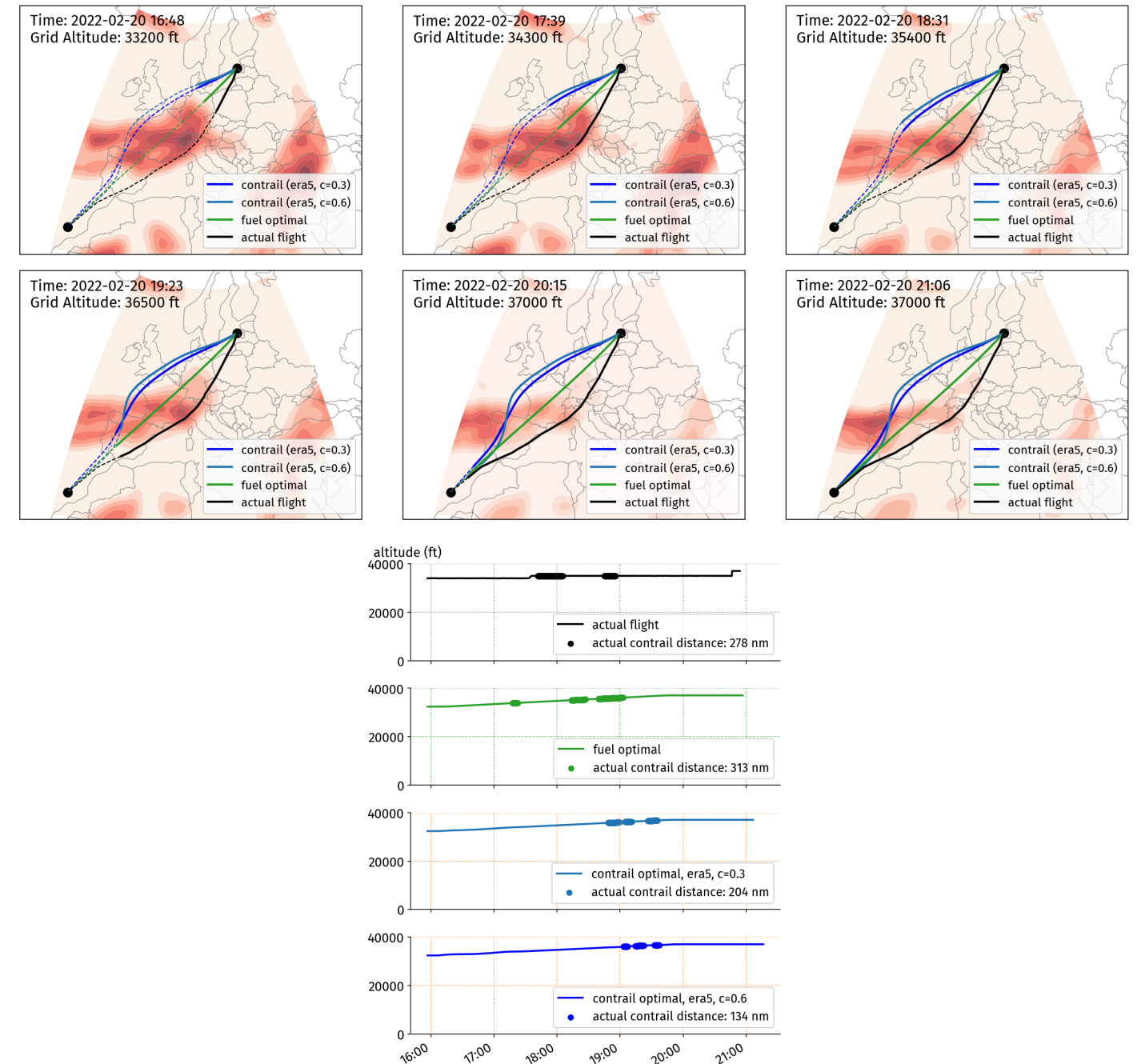
The weather-derived cost grid from the previous slide becomes `df_cost`. The optimizer then balances gridded contrail cost against CO2 from fuel burn.

This cell is disabled here because the full grid solve is too heavy for a lecture render.

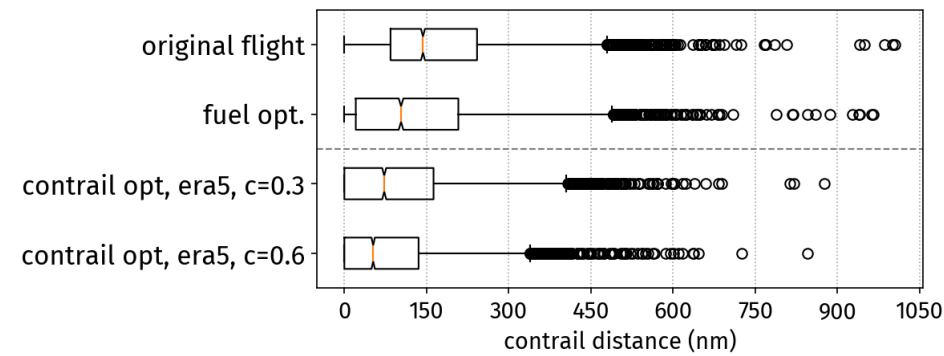
```
1 import openap
2 import opentop as top
3
4 optimizer = top.Cruise("A320", "EHAM", "LGAV", m0=0.85)
5 flight_fuel_optimal = optimizer.trajectory(objective="fuel")
6 interpolant = top.tools.interpolant_from_dataframe(df_cost)
7
8 def objective(x, u, dt, **kwargs):
9     vtas = openap.casadi.aero.mach2tas(u[0], x[2])
10    contrail_cost = optimizer.obj_grid_cost(
11        x, u, dt, n_dim=3, time_dependent=False, **kwargs
12    ) * vtas * 1e-3
13    co2_cost = optimizer.obj_fuel(x, u, dt, **kwargs) * 7.03e-15
14    return contrail_cost + co2_cost
```

Fuel versus contrail trade-off

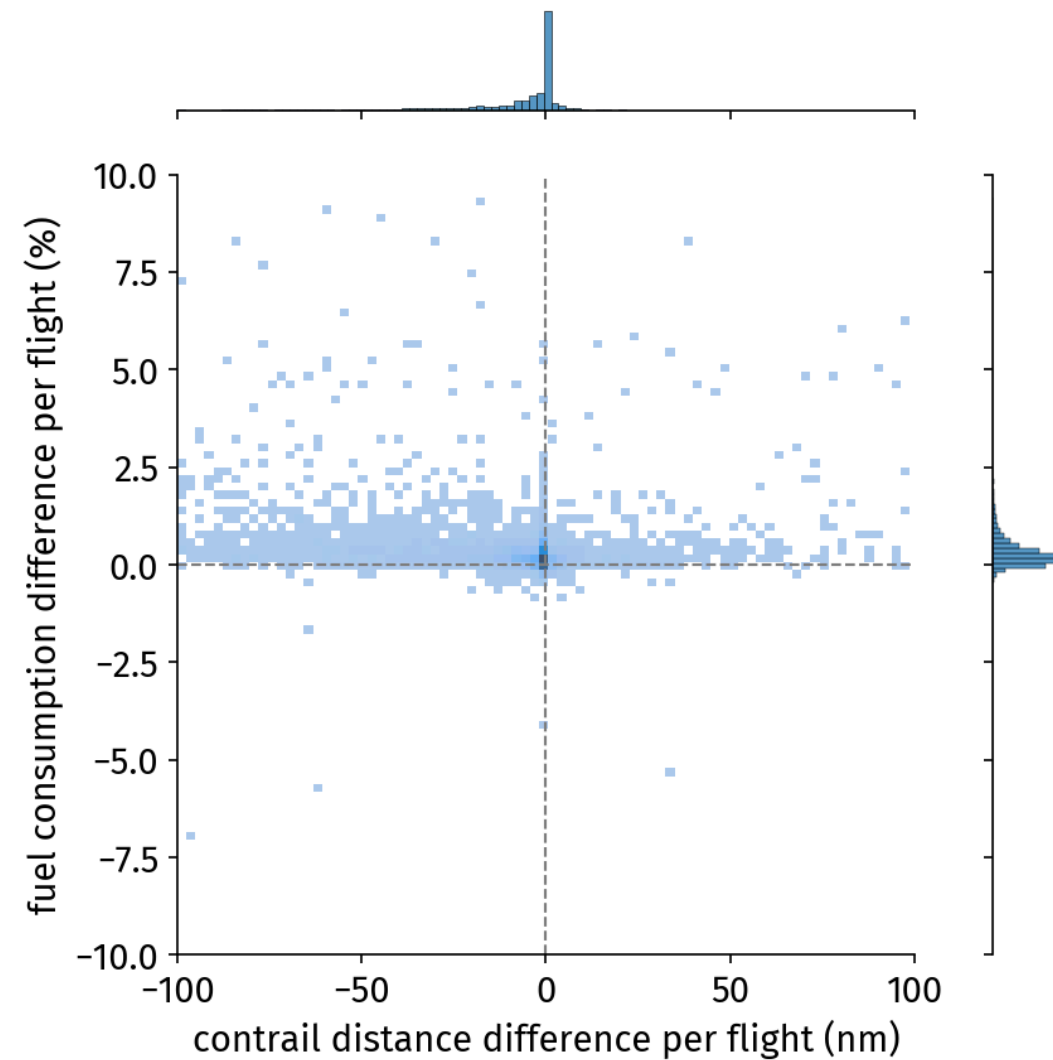
- Compare original flight, fuel-optimal flight, and contrail-aware variants.
- A higher contrail penalty can reduce contrail distance, but the fuel penalty and route deviation must be reported together.
- Vertical profiles: contrail-aware trajectories often shift altitude to avoid persistent regions.
- The challenge is whether avoidance is robust, affordable, and operationally acceptable.



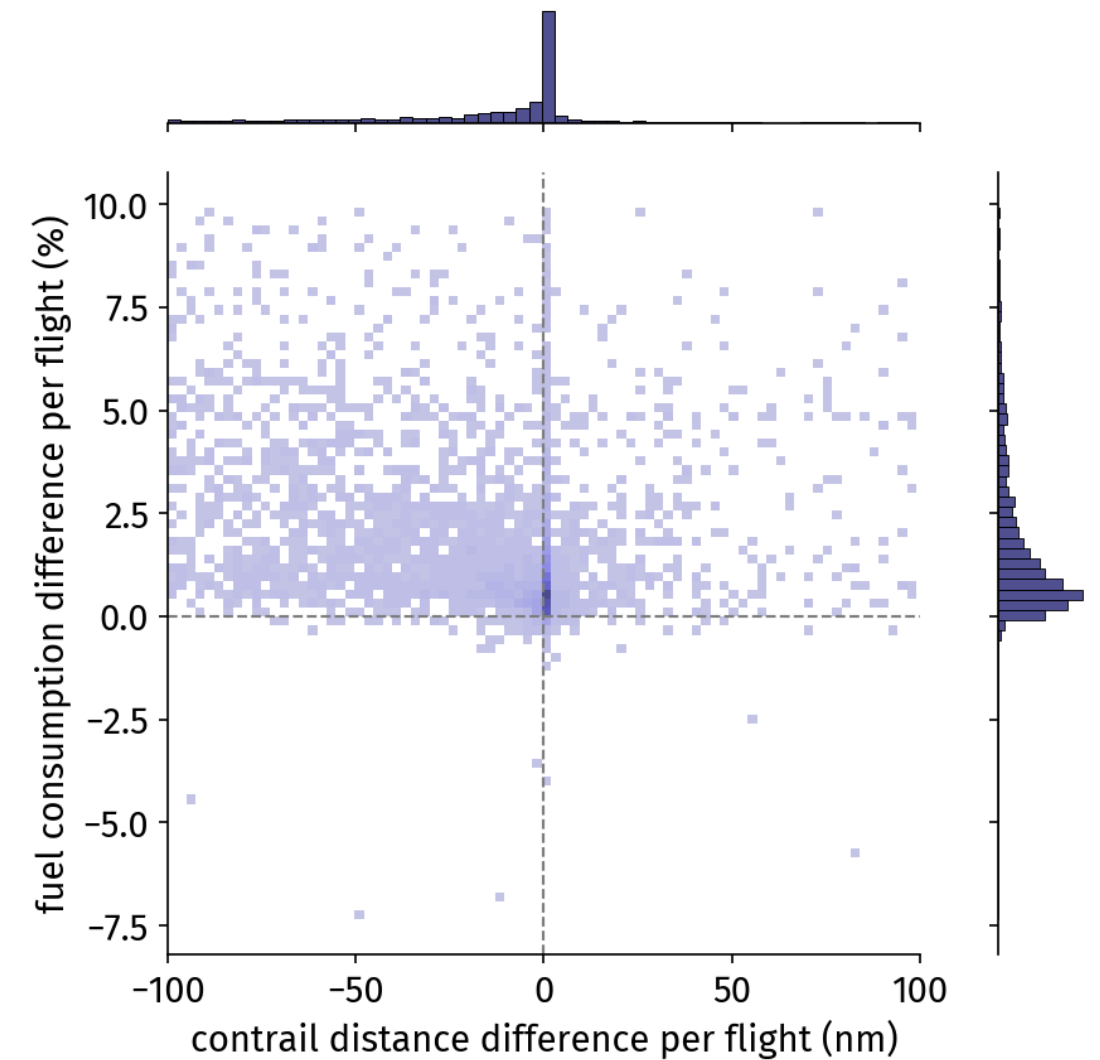
Aggregate result from one day of flights



Contrail distance for original, fuel-optimal, and contrail-aware trajectories



Fuel and contrail trade-off, $c = 0.3$

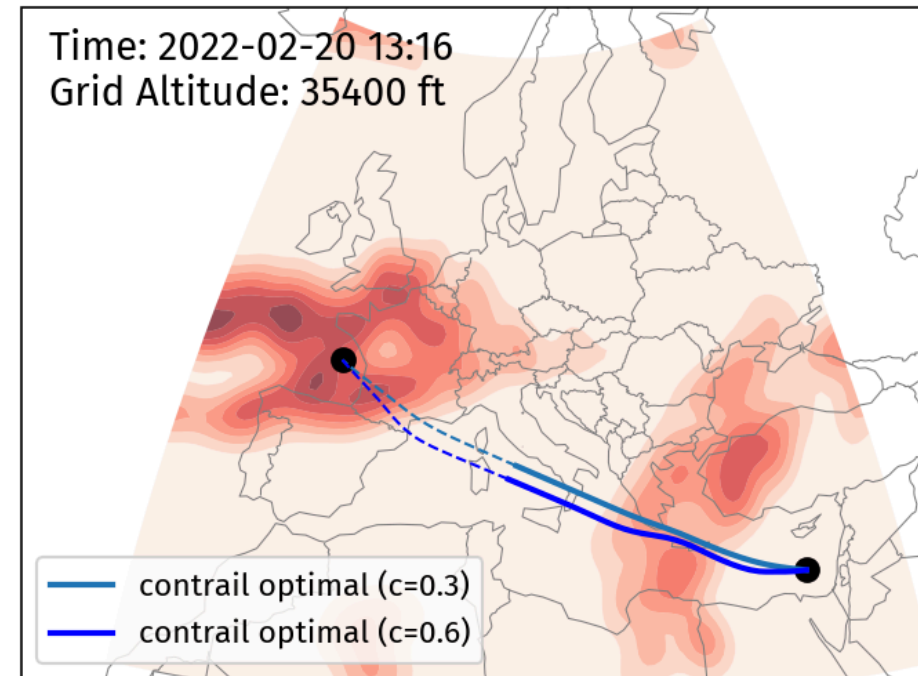


Fuel and contrail trade-off, $c = 0.6$

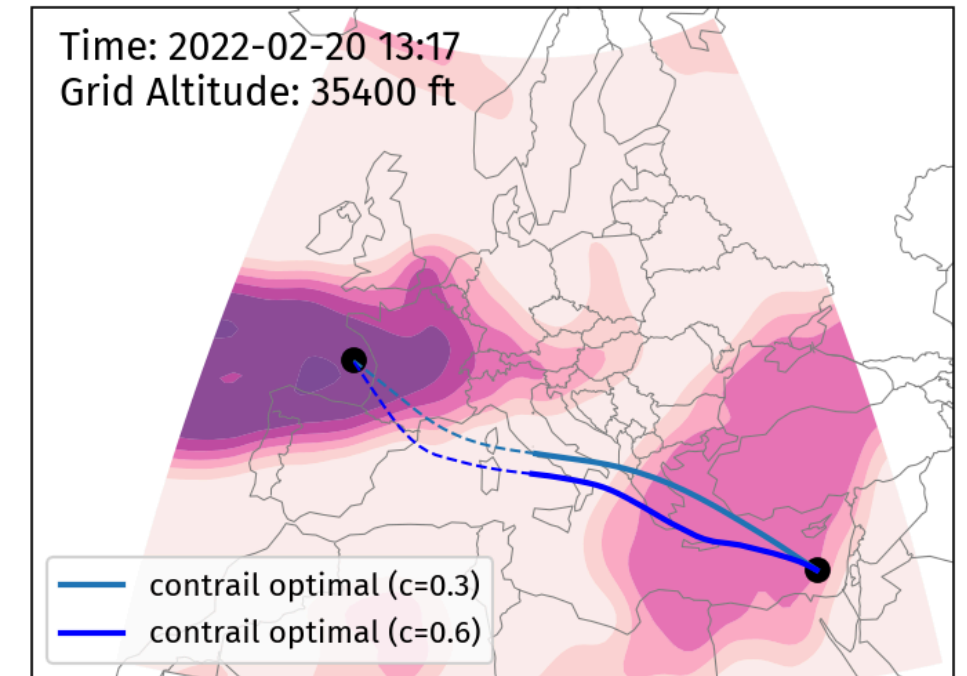
Forecast versus reanalysis

- Operational decisions must use forecast data.
- Retrospective evaluation often uses reanalysis.
- If forecast and reanalysis disagree, optimized decisions can be misplaced.
- This is why contrail-routing studies must separate evaluation data from decision data.

ERA5 reanalysis



ARPEGE forecast

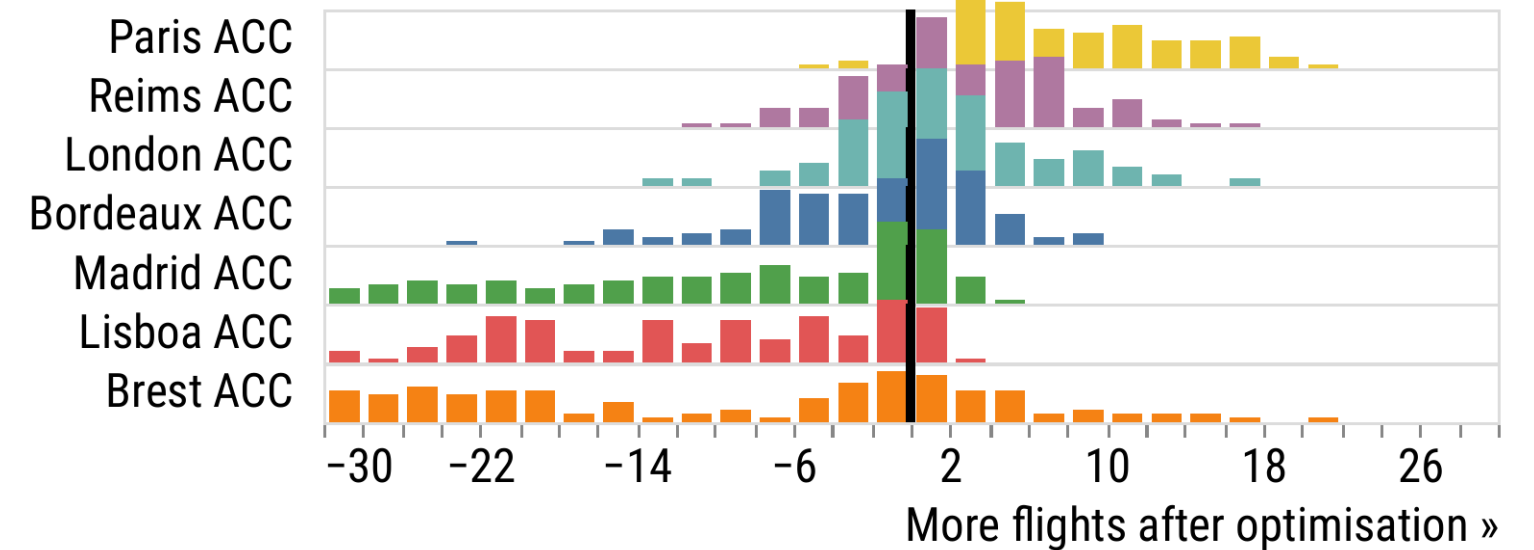


Contrail optimal trajectories obtained from ERA5 reanalysis data (left) and ARPEGE forecast data (right) over Europe on 20 February 2022

Capacity changes when many aircraft reroute

- A single contrail-avoidance trajectory may look feasible.
- Many such trajectories can shift demand into other sectors.
- Capacity and fairness become part of the climate objective.

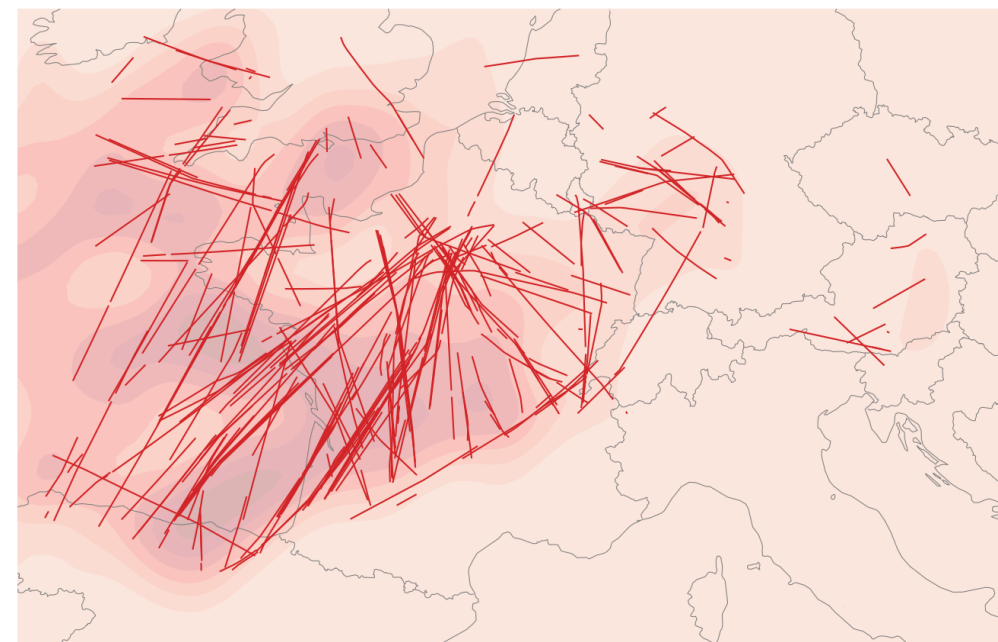
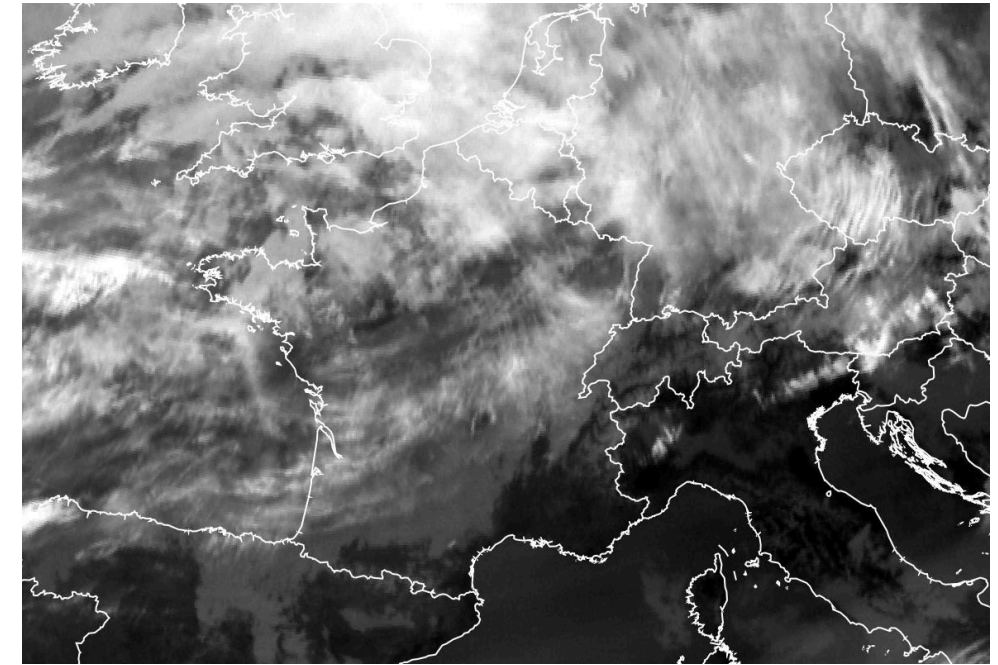
Contrail-optimal ($c=0.6$) trajectories



Capacity impact for contrail-aware trajectories

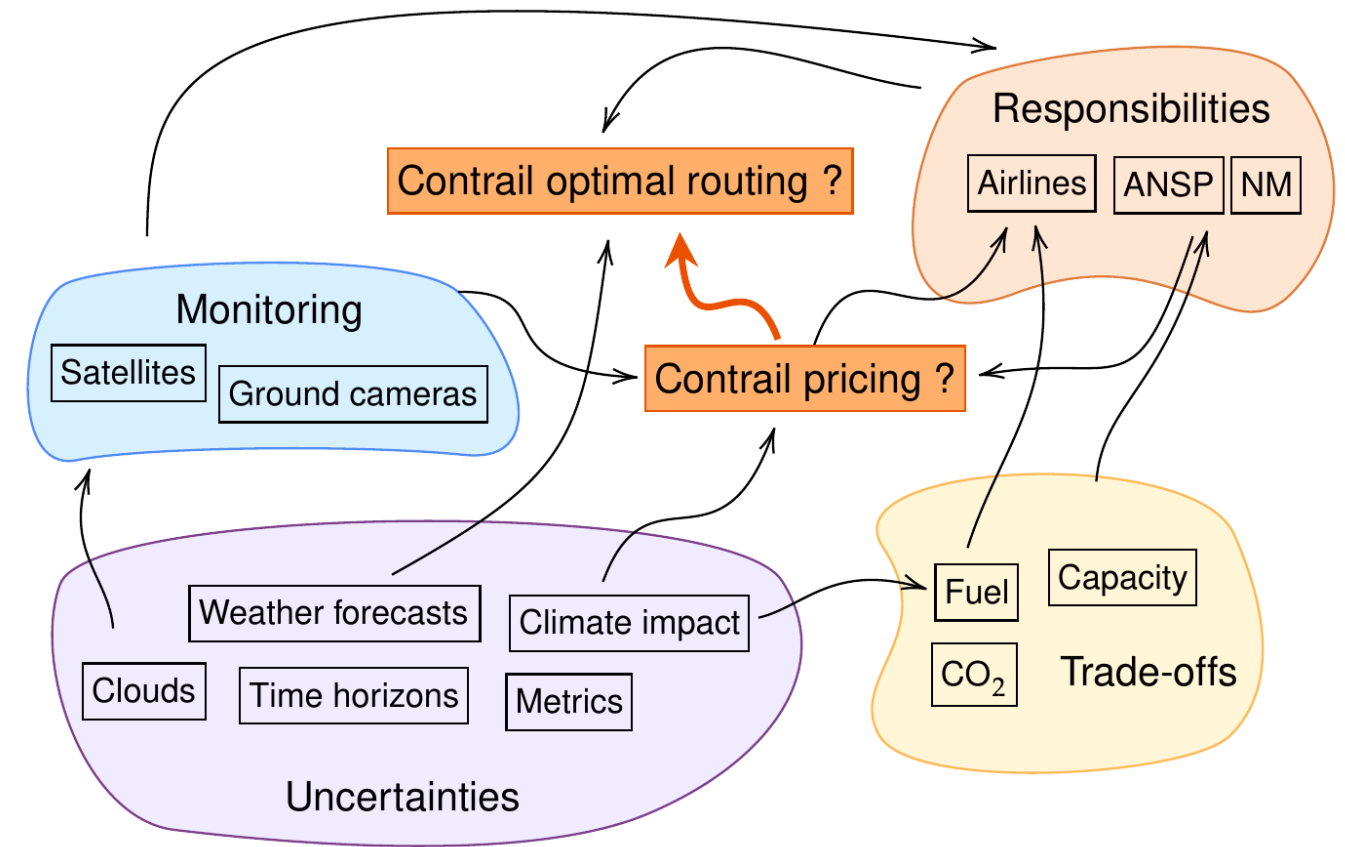
Satellite check: what happened that day?

- Satellite observations provide an external check on model-based predictions.
- They also reveal the limits of simple formation estimates.
- Future workflows need data fusion: surveillance, meteorology, satellite, and ground cameras.



Who should be responsible?

- Airlines control fuel and operating costs.
- ANSPs and network managers control flow and capacity.
- Passengers and society experience costs and benefits differently.
- Weather uncertainty and monitoring affect accountability.
- **OPTIMAL** (TU Delft - France Initiative) extends this into shared decision-making and operational feasibility.
- **FairSky** (SESAR ER3) addresses fairness when climate-optimal routing changes costs and capacity.



Operational challenges around contrail-optimal routing

Takeaways

The reusable workflow

1. **Get trajectories:** OpenSky and pyopensky.
2. **Estimate fuel and emissions:** OpenAP.
3. **Add weather and contrail conditions:** FastMeteo and OpenAP contrail tools.
4. **Compare decisions through optimization:** use [opentop](#) to test fuel, contrail, forecast, capacity, and responsibility trade-offs.

Links

Resource	Link
Lecture repository	https://github.com/junzis/isa-research-school-2026
OpenAP handbook	https://openap.dev/
pyopensky	https://mode-s.org/pyopensky/
fastmeteo	https://github.com/open-aviation/fastmeteo
OpenSky Network	https://opensky-network.org/
contrail-or-not	https://github.com/junzis/contrail-or-not
Quarto RevealJS	https://quarto.org/docs/presentations/revealjs/

open-aviation

🔍

👤

+

🔄

🔗

📧

🏠 Overview

📁 Repositories 19

📁 Projects

📦 Packages

👥 Teams

👤 People 5

📊 Insights

⋮

Open Aviation Science

👤 36 followers

🔗 <https://journals.open.tudelft.nl/joas>

Unfollow

readme.md

✎

@open-aviation | [Journal of Open Aviation Science](#) | [Book](#) | [Observable](#) | [Twitter](#)

This organisation hosts several open projects, including:

- repositories supporting the [Journal of Open Aviation Science](#);
- the source files to [A journey through aviation data](#), an open-access book;
- [pyopensky](#) library for accessing OpenSky historical database;
- more open-sources projects related to aviation

Read also about aviation related stories supported by open-source code and data:
<https://observablehq.com/@openaviation>

View as: Public

You are viewing the README and pinned repositories as a public user.

You can [hide the tasks we've suggested](#) on this page and bring them back later.

Discussions

Set up discussions to engage with your community!

[Turn on discussions](#)

Repositories

Open aviation software ecosystem